ENTORNOS DE DESARROLLO

1º DESARROLLO DE APLICACIONES WEB

TEMA 5.
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE COMPORTAMIENTO

ÍNDICE DE CONTENIDOS:

- Diagramas de comportamiento.

- Diagramas de casos de uso
    - Actores
    - Casos de uso.
    - Relaciones en los diagramas de casos de uso
        - Interacción o asociación.
        - Generalización
        - Extensión
        - Inclusión
    - Elaboración de casos de uso.
    - Escenarios de casos de uso

- Diagramas de secuencia
    - Representación de objetos y linea de vida.
    - Invocación de métodos.
    - Iteraciones y condicionales.
    - Elaboración de un diagrama de secuencia

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE COMPORTAMIENTO

Un diagrama de clases nos da información **estática:** qué clases lo componen, cómo se relacionan, las jerarquías entre ellos y tanto el estado (atributos) como comportamiento (métodos) de las clases y objetos.

La versión **dinámica** del sistema se representa con diagramas de comportamiento: cómo va a actuar el sistema con el usuario, cuál es la secuencia de ejecución, en qué orden van a ejecutarse los métodos, el ciclo de vida de los objetos, etc.

Los diagramas de comportamiento pueden ser:

- Diagramas de casos de uso.
- Diagramas de actividad.
- Diagramas de máquinas de estado.
- Diagramas de interacción.
- Diagramas de secuencia.
- Diagramas de comunicación.
- Diagramas de interacción.
- Diagramas de tiempo.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

Los diagramas de casos de uso son un elemento fundamental del análisis de un sistema desde la perspectiva de la orientación a objetos porque nos ayuda a determinar **QUÉ puede hacer cada tipo diferente de usuario con el sistema**, en una forma que cualquiera pueda entender.

Los diagramas de casos de uso documentan el **comportamiento de un sistema desde el punto de vista del usuario.**

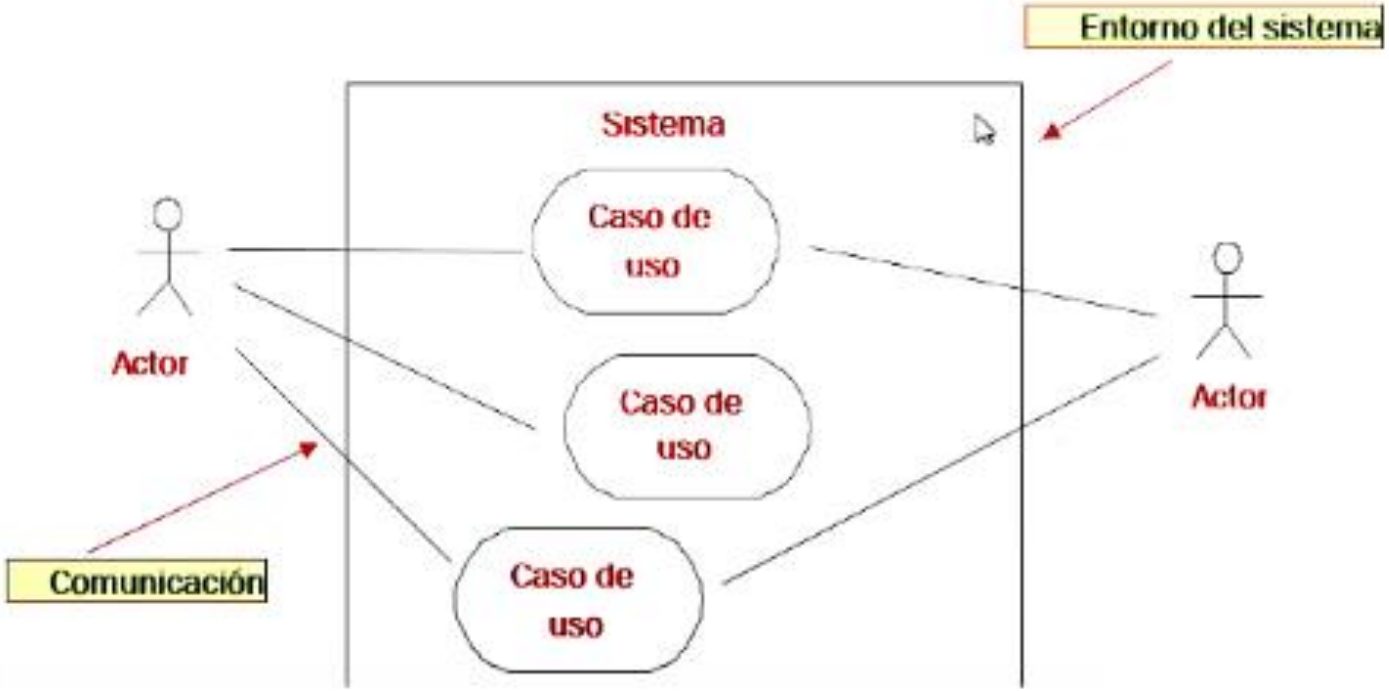
Un diagrama de casos de uso es una **visualización gráfica de los requisitos funcionales del sistema**, formado por casos de uso (se representan como elipses) y los actores que interactúan con ellos (se representan como monigotes). Su ventaja principal es la facilidad para interpretarlos, lo que hace que sean especialmente útiles en la comunicación con el cliente.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ELEMENTOS QUE LOS COMPONEN

Sistema



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ELEMENTOS QUE LOS COMPONEN

Actores

Un actor representa un tipo de usuario del sistema. Se entiende como usuario cualquier cosa externa que interactúa con el sistema. No tiene por qué ser un ser humano, puede ser otro sistema informático o unidades organizativas o empresas.

Siempre hay que intentar independizar los actores a partir de la forma en que se interactúa con el sistema. Un mismo usuario del sistema puede interpretar diferentes roles según la operación que esté ejecutando. Pues bien, cada uno de estos roles representará un actor diferente, aunque en realidad hablemos de la misma persona. Suele ser útil mantener una lista de los usuarios reales para cada actor.

Es posible que haya casos de uso que no sean iniciados por ningún usuario, o algún otro elemento software, en ese caso se puede crear un actor "Tiempo" o "Sistema".

¿Cómo encontrar los actores?

- Identificar los usuarios y administradores de la aplicación.
- Identificar los roles de los usuarios de la aplicación.
- Identificar los sistemas y/o aplicaciones que interactúan con la aplicación.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ELEMENTOS QUE LOS COMPONEN

Casos de uso.

Un caso de uso especifica una secuencia de acciones que el sistema puede llevar a cabo, y que producen un resultado observable de valor para un actor concreto. El conjunto de casos de uso forma el **"comportamiento requerido"** de un sistema.

El objetivo principal de elaborar un diagrama de casos de uso no es crear el diagrama en sí, sino la descripción de cada caso, ya que esto es lo que ayuda al equipo de desarrollo a crear el sistema a posteriori.

¿Cómo encontrar los casos de uso por cada actor?

- Identificar las tareas principales de los actores.
- Identificar la información a consultar / modificar / eliminar.
- Identificar la información a suministrar:
  - Del actor al sistema
  - Del sistema al actor

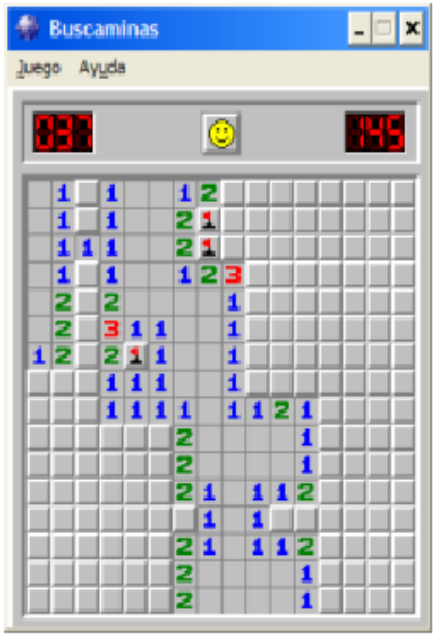
Un caso de uso siempre es iniciado por un actor externo.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ELEMENTOS QUE LOS COMPONEN

Ejemplo:

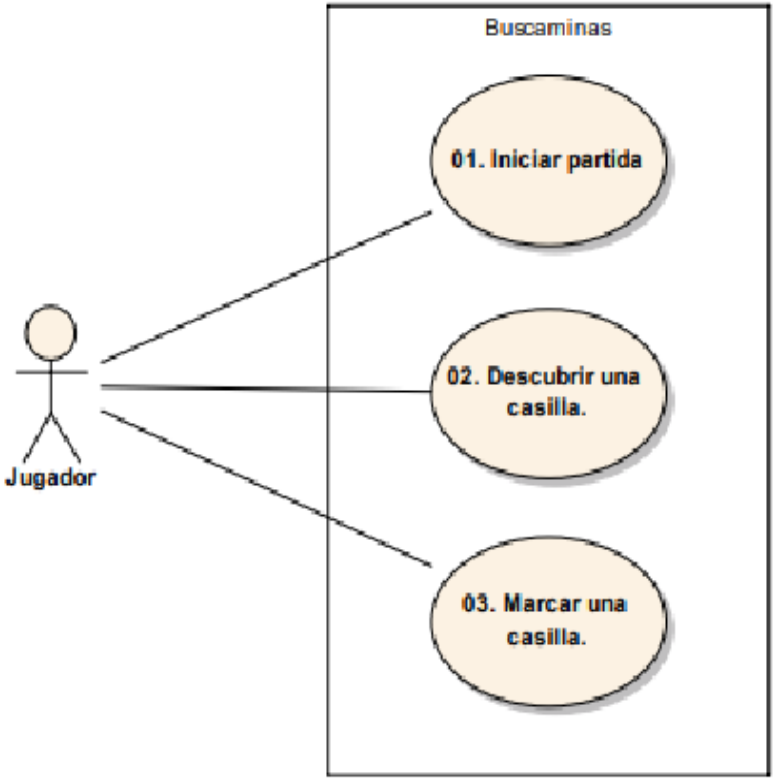


¿Qué casos de uso identificamos?

- Iniciar una nueva partida.
- Descubrir una casilla.
- Marcar una casilla.

¿Quién realiza estos casos de uso?

- El jugador.



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

RELACIONES EN LOS DIAGRAMAS DE CASOS DE USO

Existen diferentes tipos de relaciones entre elementos:

- **Asociación**: representa la relación entre el actor que lo inicia y el caso de uso.
- **Inclusión**: se utiliza cuando queremos dividir una tarea de mayor envergadura en otras más sencillas, que son utilizadas por la primera. Son muy útiles cuando es necesario reutilizar tareas.
- **Extensión**: se utiliza para representar relaciones entre un caso de uso que requiere la ejecución de otro en determinadas circunstancias.
- **Generalización**: se utiliza para representar relaciones de **herencia** entre casos de uso o actores.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

RELACIONES EN LOS DIAGRAMAS DE CASOS DE USO

Interacción o asociación.

Hay una asociación entre un actor y un caso de uso si el actor interactúa con el sistema para llevar a cabo el caso de uso o para iniciarlo.

Una asociación se representa mediante un linea continua que une un actor con un caso de uso. Por ejemplo, un usuario de un sistema de venta por Internet puede hacer un pedido, lo que se representa del siguiente modo:

DIAGRAMAS DE COMPORTAMIENTO

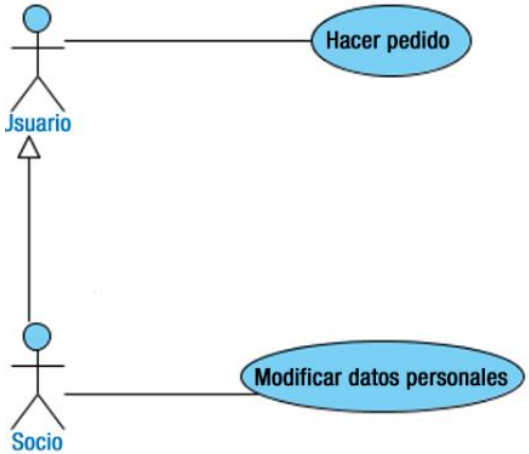
DIAGRAMAS DE CASOS DE USO

RELACIONES EN LOS DIAGRAMAS DE CASOS DE USO

Generalización

Es posible que, igual que con los diagramas de clases, existan casos de uso que tengan comportamientos semejantes a otros que los modifican o completan de alguna manera. El caso base se define de forma abstracta y los hijos heredan sus características añadiendo sus propios pasos o modificando alguno. Normalmente la herencia se utiliza menos en diagramas de casos de uso que en diagramas de clases.

Por ejemplo, el usuario del sistema de venta por Internet puede a su vez darse de alta en la página web para que tengan sus datos registrados a la hora de hacer el pedido, en este caso el usuario es la generalización del socio. Ambos actores pueden hacer un pedido, pero solo el socio puede modificar sus datos en el sistema.



DIAGRAMAS DE COMPORTAMIENTO

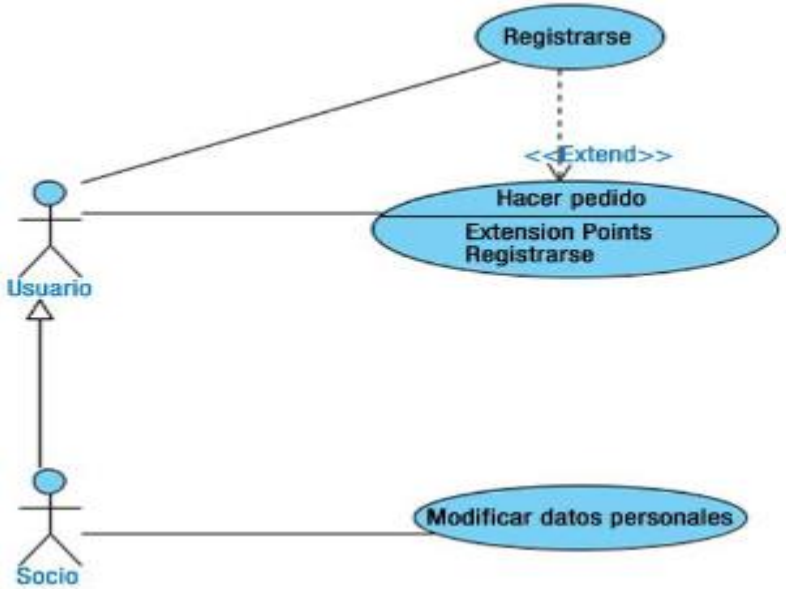
DIAGRAMAS DE CASOS DE USO

RELACIONES EN LOS DIAGRAMAS DE CASOS DE USO

Extensión

Se utiliza una relación entre dos casos de uso de tipo "**extends**" cuando se desea especificar que **el comportamiento de un caso de uso es diferente dependiendo de ciertas circunstancias.** Otra forma de verlo es decir que se utiliza cuando existe una parte del caso de uso que se ejecuta sólo en determinadas ocasiones, pero no es imprescindible para su completa ejecución.

Por ejemplo, cuando un usuario hace un pedido si no es socio se le ofrece la posibilidad de darse de alta en el sistema en ese momento, pero puede realizar el pedido aunque no lo sea.



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

RELACIONES EN LOS DIAGRAMAS DE CASOS DE USO

Inclusión

Se incluye una relación entre dos casos de uso de tipo "**include**" cuando **la ejecución del caso de uso incluido se da en la rutina normal del caso que lo incluye.**

Esta relación es muy útil cuando se desea especificar algún comportamiento común en dos o más casos de uso.

Por ejemplo, a la hora de hacer un pedido se debe buscar la información de los artículos para obtener el precio, es un proceso que necesariamente forma parte del caso de uso, sin embargo también forma parte de otros, como son el que visualiza el catálogo de productos y la búsqueda de un artículo concreto, y dado que tiene entidad por sí solo se separa del resto de casos de uso y se incluye en los otros tres.

Las ventajas de esta asociación son:

- Las descripciones de los casos de uso son más cortas y se entienden mejor.
- La identificación de funcionalidad común puede ayudar a descubrir el posible uso de componentes ya existentes en la implementación.

Las desventajas son:

DIAGRAMAS DE COMPORTAMIENTO

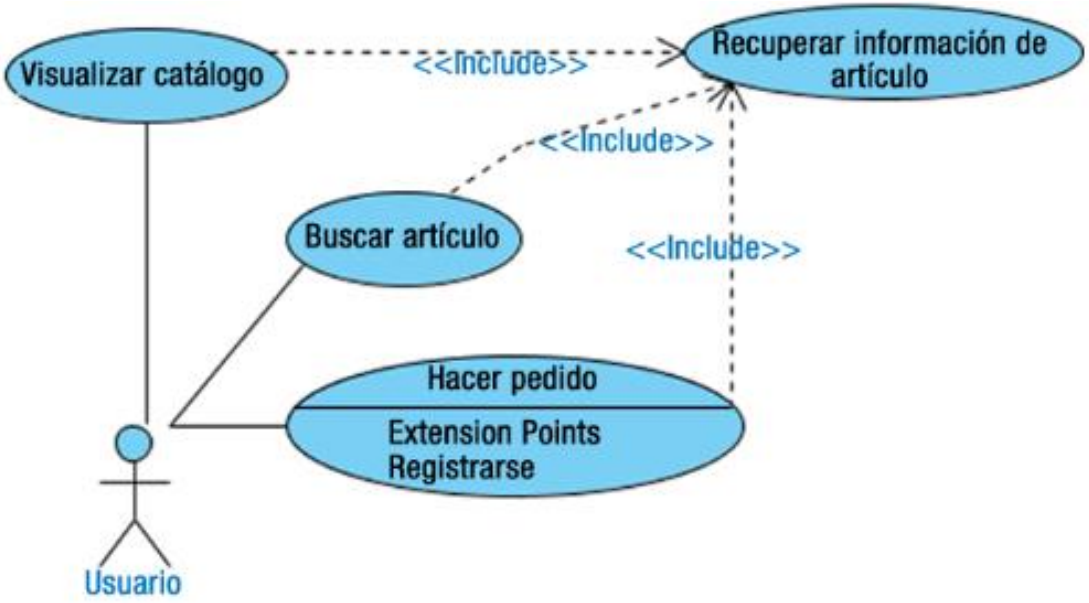
DIAGRAMAS DE CASOS DE USO

RELACIONES EN LOS DIAGRAMAS DE CASOS DE USO

Las desventajas son:

- La inclusión de estas relaciones hace que los diagramas sean más difíciles de leer, sobre todo para los clientes.

Cuando usamos relaciones de inclusión o extensión no podemos olvidar que los casos de uso extendidos o incluidos deben cumplir con las características propias de un caso de uso, es decir, deben representar un flujo de actividad completo desde el punto de vista de lo que un actor espera que el sistema haga por él, así como no utilizar estas herramientas sólo para descomponer un caso de uso de envergadura en otros más pequeños, piedra angular del diseño estructurado y no del orientado a objetos.



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ELABORACIÓN DE LOS CASOS DE USO

Siempre que elaboramos un diagrama, del tipo que sea, hacemos un proceso de abstracción por el cual representamos elementos de la realidad esquemáticamente. En el caso de un diagrama de casos de uso lo que hacemos es abstraer la realidad en un dibujo, en el que representamos qué cosas pueden hacerse en nuestro sistema y quién las va a hacer.

Los diagramas deben incluir información suficiente para que el equipo de desarrollo tome las decisiones más adecuadas para desarrollar un software que cumpla con los requerimientos y deben servir en la fase de implementación en un lenguaje orientado a objetos.

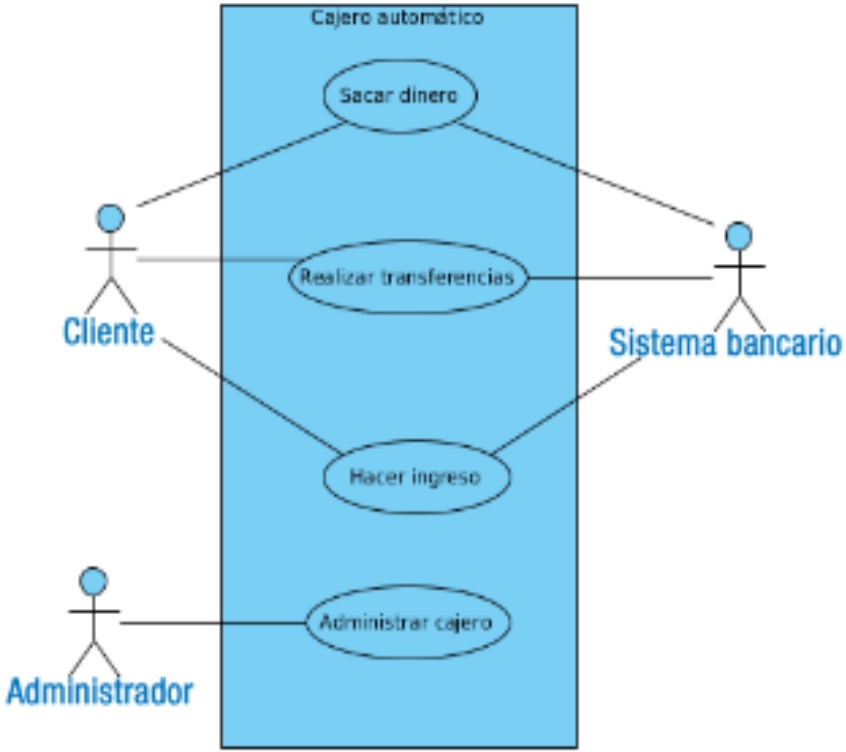
Para elaborar el diagrama debemos partir de una descripción lo más detallada posible del problema a resolver y trataremos de detectar quien interactúa con el sistema, para obtener los actores. A continuación buscaremos qué tareas realizan estos actores para determinar los casos de uso más genéricos. El siguiente paso es refinar el diagrama analizando los casos de uso más generales para detectar casos relacionados por inclusión (se detectan fácilmente cuando aparecen en dos o más casos de uso generales), extensión y generalización. Al diagrama generado se le denomina diagrama frontera.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ELABORACIÓN DE LOS CASOS DE USO

Se conoce como **diagrama frontera** al diagrama de casos de uso que incluye todos los casos de uso genéricos del sistema, que podrán ser desglosados después en nuevos diagramas de casos de uso que los describan si es necesario. Se especifica enmarcando los casos de uso en un recuadro, que deja a los actores fuera.



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ESCENARIOS DE CASOS DE USO

Un caso de uso debe especificar un comportamiento deseado, pero no imponer cómo se llevará a cabo ese comportamiento, es decir, debe decir QUÉ pero no CÓMO. Esto se realiza utilizando escenarios que son casos particulares de un caso de uso.

Un **escenario** es una ejecución particular de un caso de uso que se describe como una secuencia de eventos. Un caso de uso es una generalización de un escenario.

Por ejemplo, para el caso de uso hacer pedido podemos establecer diferentes escenarios.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO

ESCENARIOS DE CASOS DE USO

Realizar pedido de unos zapatos y unas botas.

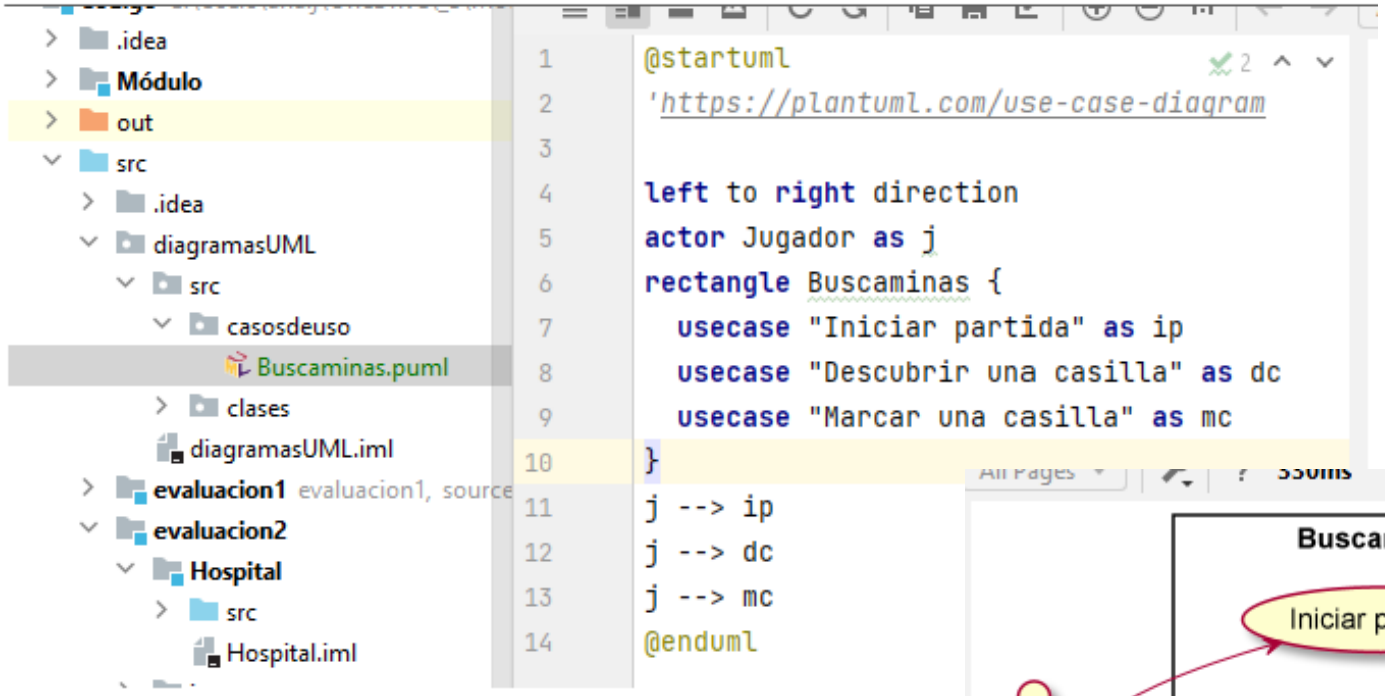
1. El usuario inicia el pedido.
2. Se crea el pedido en estado "en construcción".
3. Se selecciona un par de zapatos "Lucía" de piel negros, del número 39.
4. Se selecciona la cantidad 1.
5. Se recupera la información de los zapatos y se modifica la cantidad a pagar sumándole 45 €.
6. Se selecciona un par de botas "Aymara" de ante marrón del número 40.
7. Se selecciona la cantidad 1.
8. Se recupera la información de las botas y se modifica la cantidad a pagar sumándole 135 €.
9. El usuario acepta el pedido.
10. Se comprueba que el usuario es, efectivamente socio.
11. Se comprueban los datos bancarios, que son correctos.
12. Se calcula el total a pagar añadiendo los gastos de envío.
13. Se realiza el pago a través de una entidad externa.
14. Se genera un pedido para el usuario con los dos zapatos que ha comprado, con el estado "pendiente"

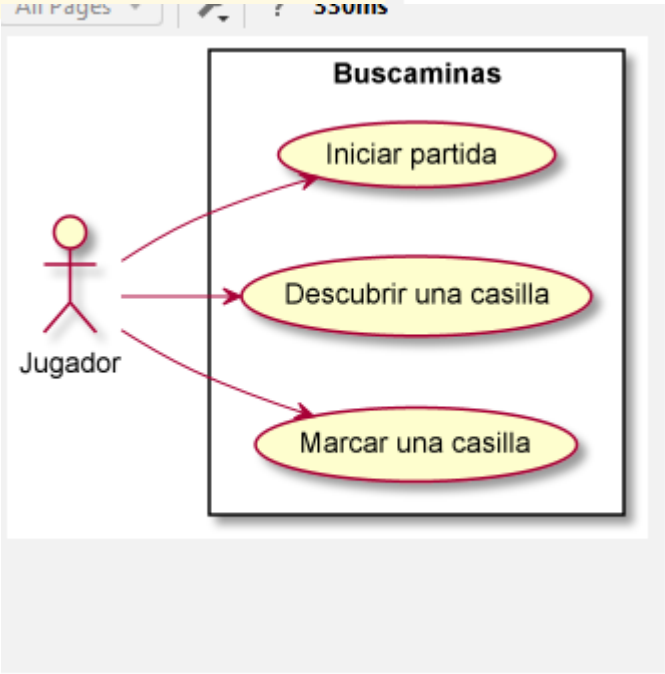
Los escenarios pueden y deben posteriormente documentarse mediante diagramas de secuencia.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO CON UML

Buscaminas



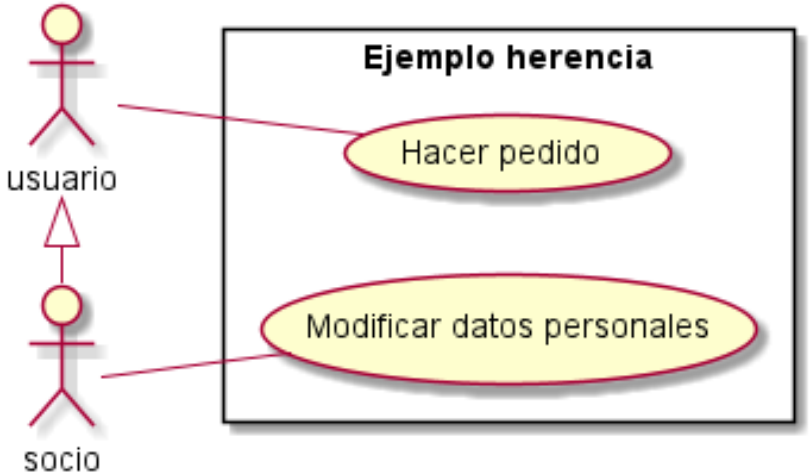


DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO CON UML

Ejemplo de generalización (herencia)

```
@startuml
'https://plantuml.com/use-case-diagram

left to right direction
actor usuario
actor socio
usuario <|- socio
rectangle "Ejemplo herencia" {
  usuario -- (Hacer pedido)
  socio -- (Modificar datos personales)

@enduml
```

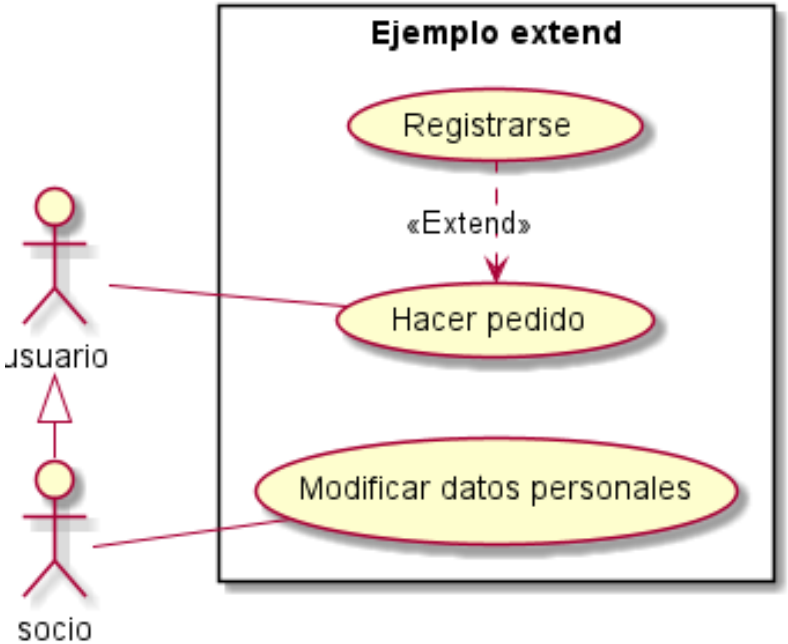


DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO CON UML

Ejemplo de extend

```
@startuml
'https://plantuml.com/use-case-diagram

left to right direction
actor usuario
actor socio
usuario <|- socio
rectangle "Ejemplo extend" {
  usuario -- (Hacer pedido)
  (Hacer pedido) <. (Registrarse): <<Extend>>
  socio -- (Modificar datos personales)

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

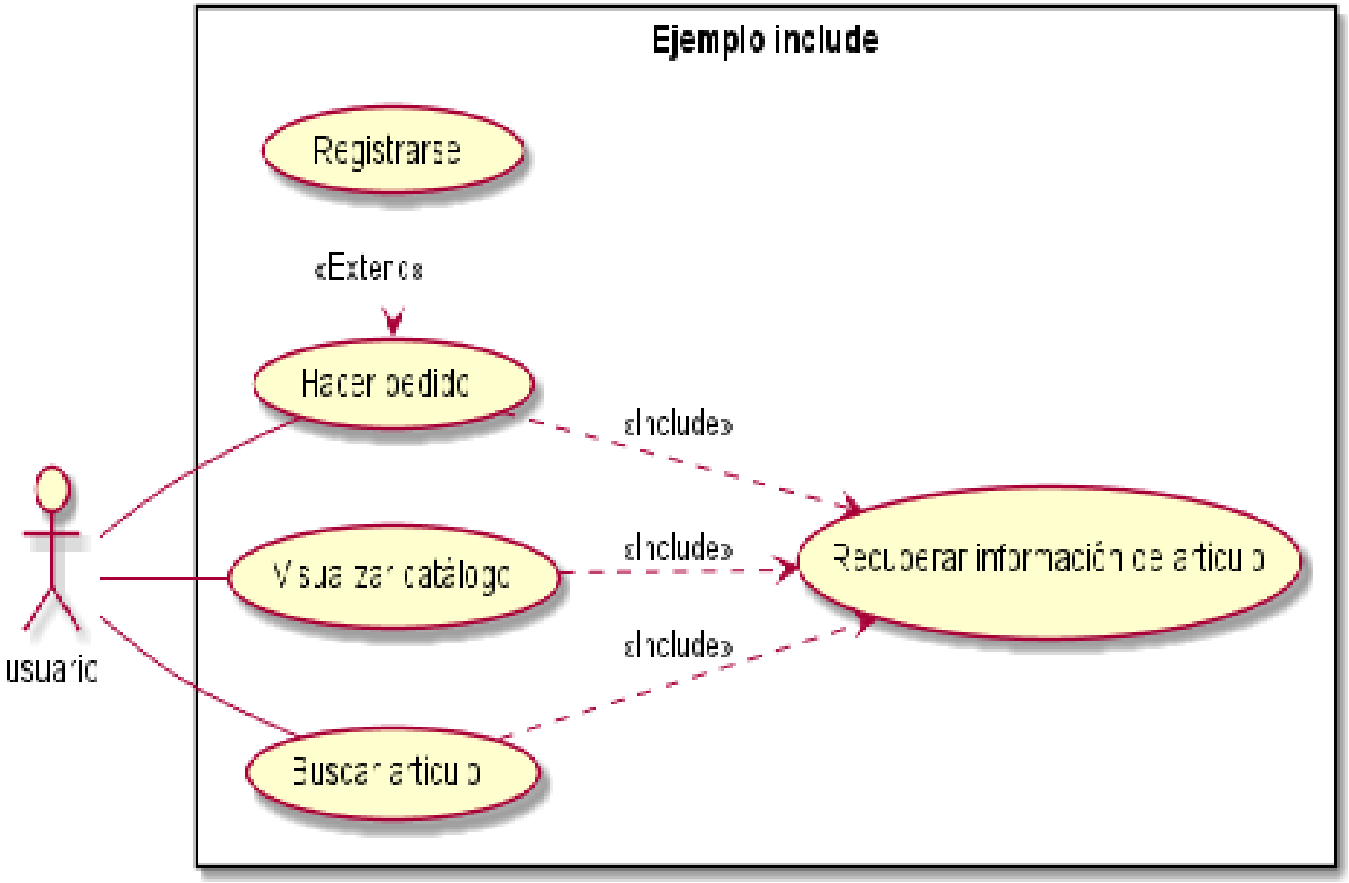
DIAGRAMAS DE CASOS DE USO CON UML

Ejemplo de include

```
@startuml
'https://plantuml.com/use-case-diagram

left to right direction
actor usuario
rectangle "Ejemplo include" {

  usuario -- (Visualizar catálogo)

  (Visualizar catálogo)..> (Recuperar información de artículo): <<Include>>

  usuario -- (Buscar artículo)

  (Buscar artículo) ..> (Recuperar información de artículo): <<Include>>

  usuario -- (Hacer pedido)

  (Hacer pedido) ..> (Recuperar información de artículo): <<Include>>

  (Hacer pedido) <. (Registrarse) : <<Extend>>

@enduml
```

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO CON UML



DIAGRAMAS DE COMPORTAMIENTO

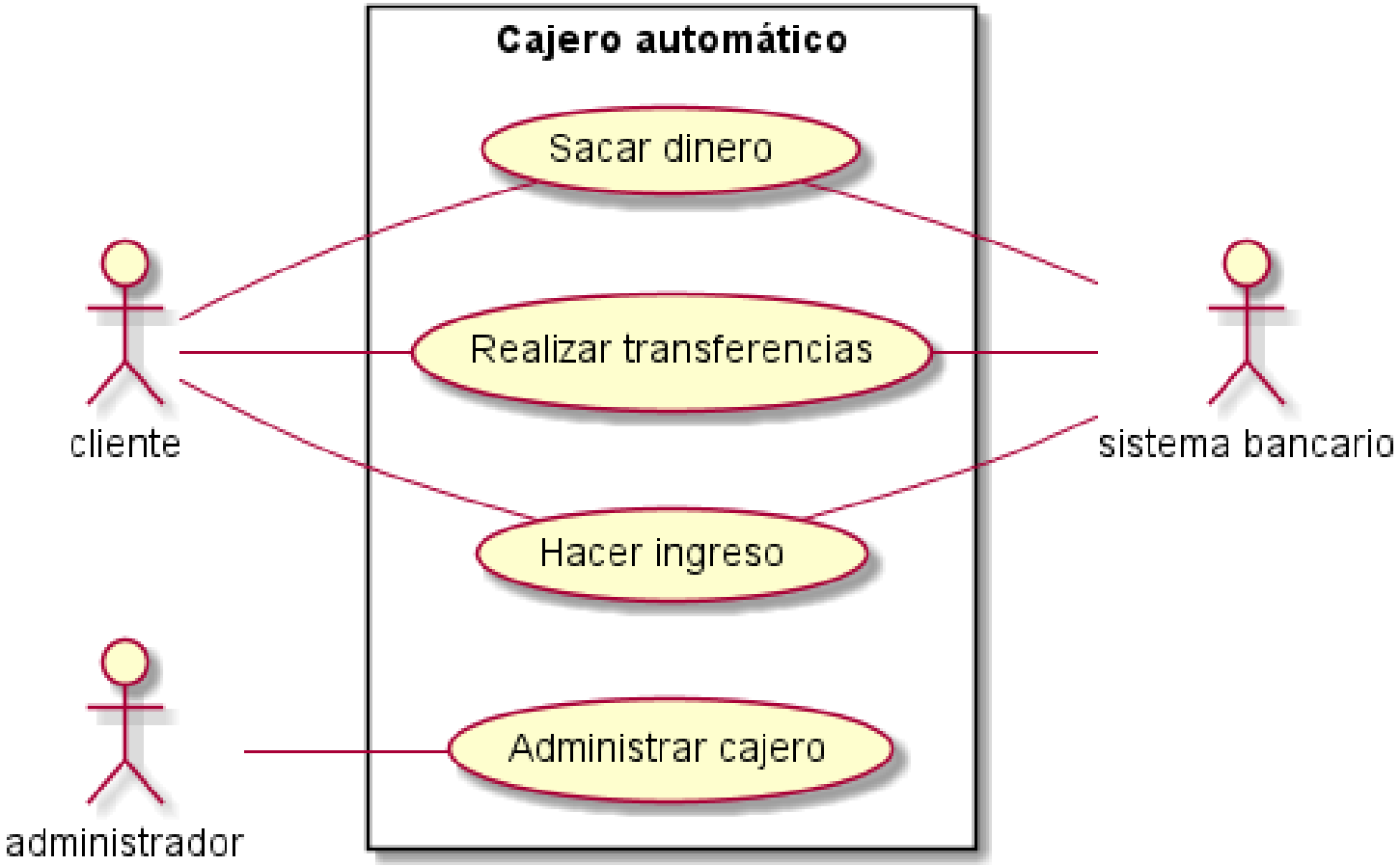
DIAGRAMAS DE CASOS DE USO CON UML

Cajero automático

```
@startuml
'https://plantuml.com/use-case-diagram

left to right direction
actor cliente
actor administrador
actor "sistema bancario" as sb
rectangle "Cajero automático" {
  cliente -- (Sacar dinero)
  cliente -- (Realizar transferencias)
  cliente -- (Hacer ingreso)
  administrador -- (Administrar cajero)
 (Sacar dinero) -- sb
 (Realizar transferencias) -- sb
 (Hacer ingreso) -- sb

}
@enduml
```

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE CASOS DE USO CON UML



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO

Ejercicio 1 – sokoban

Sokoban es un juego de varios niveles.

Cada nivel está compuesto por un jugador, cajas, repisas y muros.

El objetivo del jugador es empujar todas las cajas sobre las repisas.

Cuando esto sucede el jugador pasa al siguiente nivel.

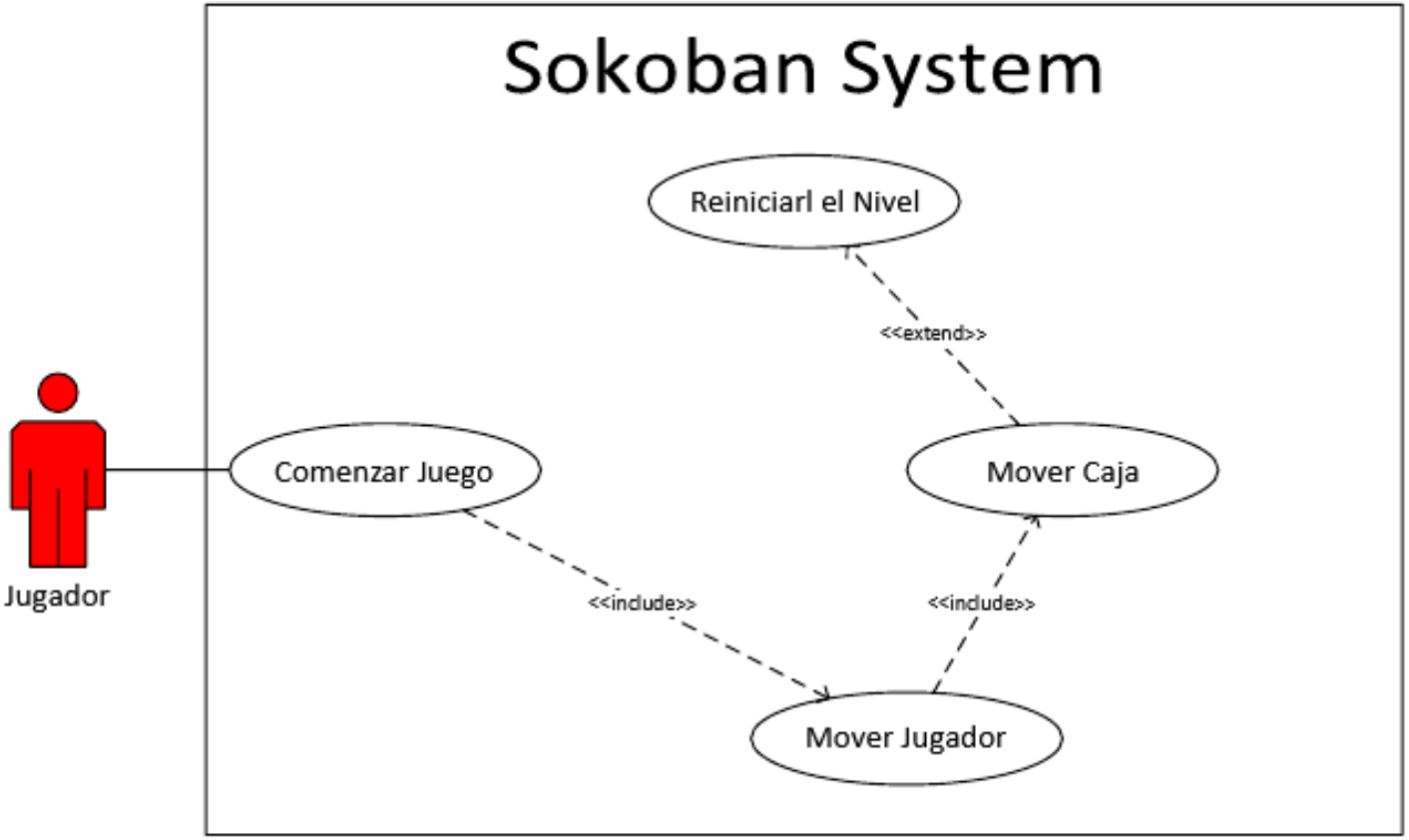
Para mover una caja, el jugador debe colocarse al lado y empujarla. Si la casilla hacia la que está empujando la caja está libre, la caja se moverá.

Si el jugador se queda bloqueado, es decir, no puede terminar el nivel, puede reiniciar el nivel perdiendo una vida.

Cuando el jugador pierde todas sus vidas la partida termina.

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO

Ejercicio 2 – sistema de cambio de grupo para asignaturas

Un sistema automático de cambio de grupos para asignaturas funciona de la siguiente manera:

El profesor da de alta una asignatura y proporciona al sistema un listado con los alumnos matriculados en dicha asignatura.

Un alumno que quiera cambiar de grupo en una asignatura puede consultar las peticiones de cambio.

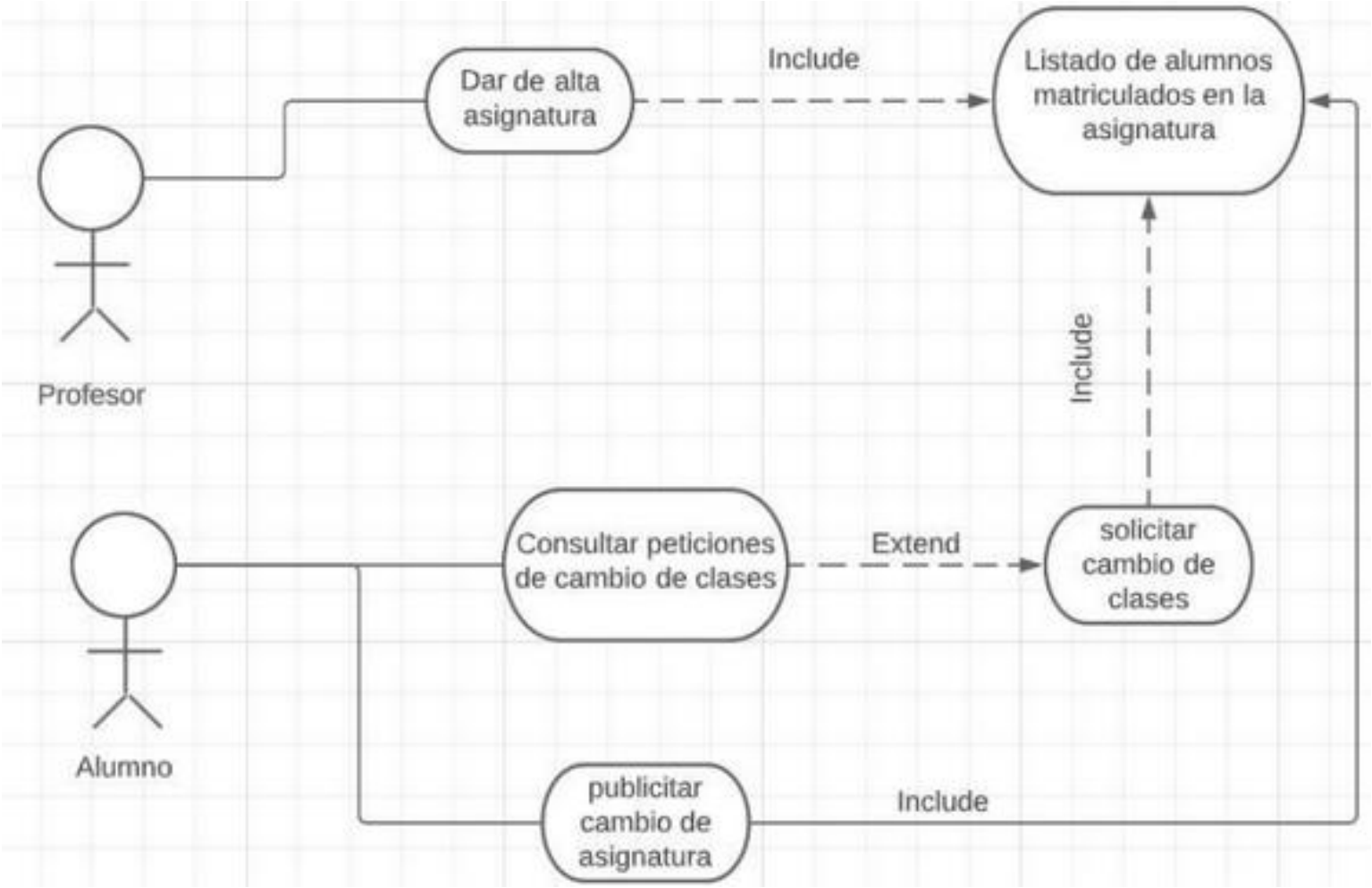
Si encuentra alguna que le interese, el alumno solicita el cambio y el sistema lo almacena.

Si no, el alumno puede dejar el cambio que desea por si a otro alumno le interesara.

Los alumnos sólo pueden consultar y publicitar cambios de las asignaturas en las que están matriculados.

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO

Ejercicio 3 – sistema de encuentros virtuales

Se desea desarrollar un sistema de encuentros virtuales (parecido a un chat).

Cuando se conecta a un servidor, un usuario puede entrar o salir de un encuentro.

Cada encuentro tiene un manager.

El manager es el usuario que ha planificado el encuentro (el nombre del encuentro, la agenda del encuentro y el moderador del encuentro).

Cada encuentro puede tener también un moderador designado por el manager.

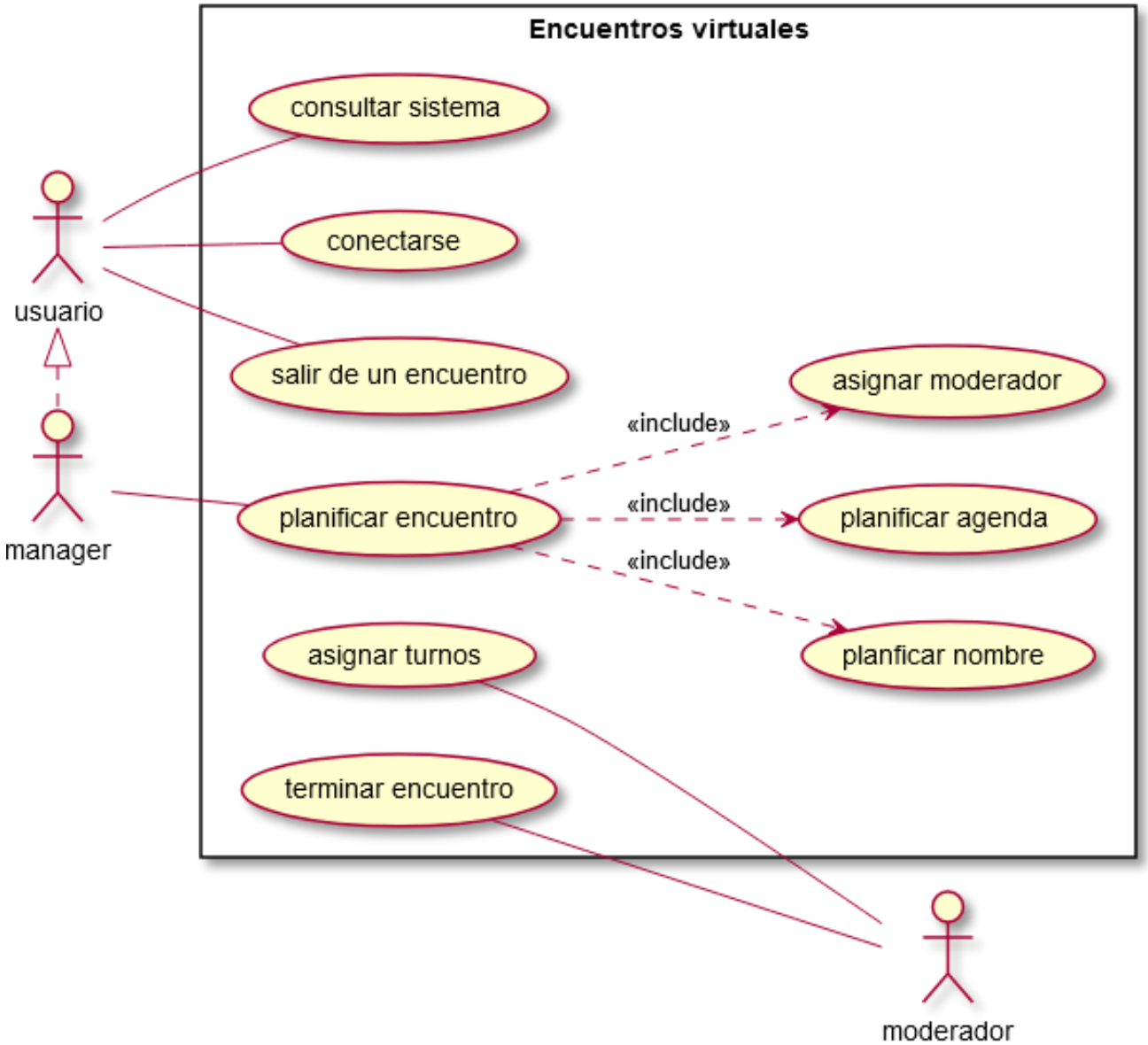
La misión del moderador es asignar los turnos de palabra para que los usuarios hablen.

El moderador también podrá dar por concluido el encuentro en cualquier momento.

En cualquier momento un usuario puede consultar el estado del sistema, por ejemplo los encuentros planeados y su información.

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO

Ejercicio 4 – bolsa

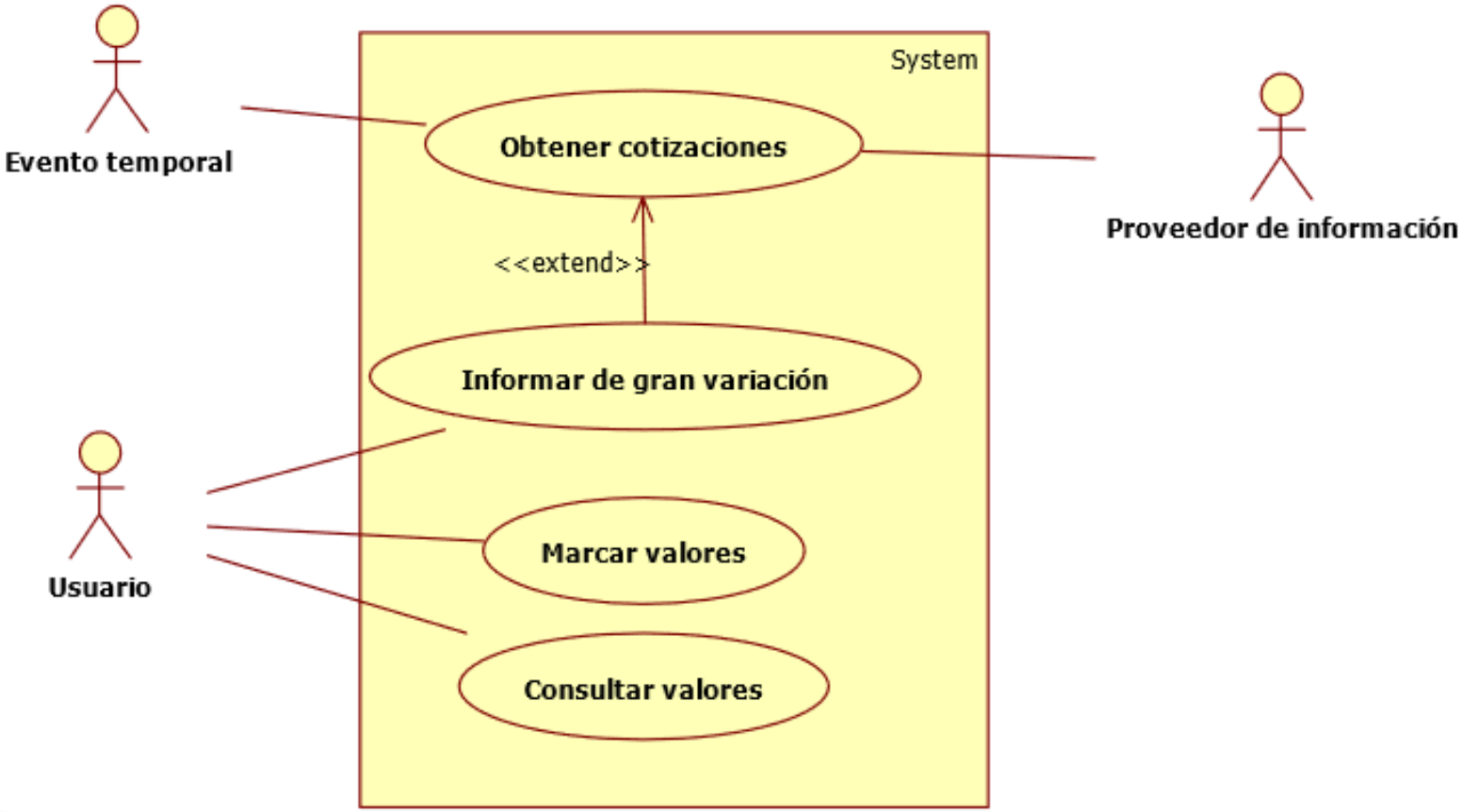
Un sistema personal de bolsa se conecta periódicamente a servidores que ofrecen información de las cotizaciones.

El sistema personal permite marcar una serie de valores para realizar un seguimiento y consultar los datos de dichos valores.

Si a la hora de actualizar las cotizaciones uno de los valores marcados presenta una gran subida o bajaa, informará al usuario de ello.

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO

Ejercicio 5 – juego de teléfono móvil

Tenemos un juego de teléfono móvil donde participan dos jugadores, cada uno con su propia terminal.

Cuando dos jugadores desean jugar, uno de ellos crea una nueva partida y el otro se conecta.

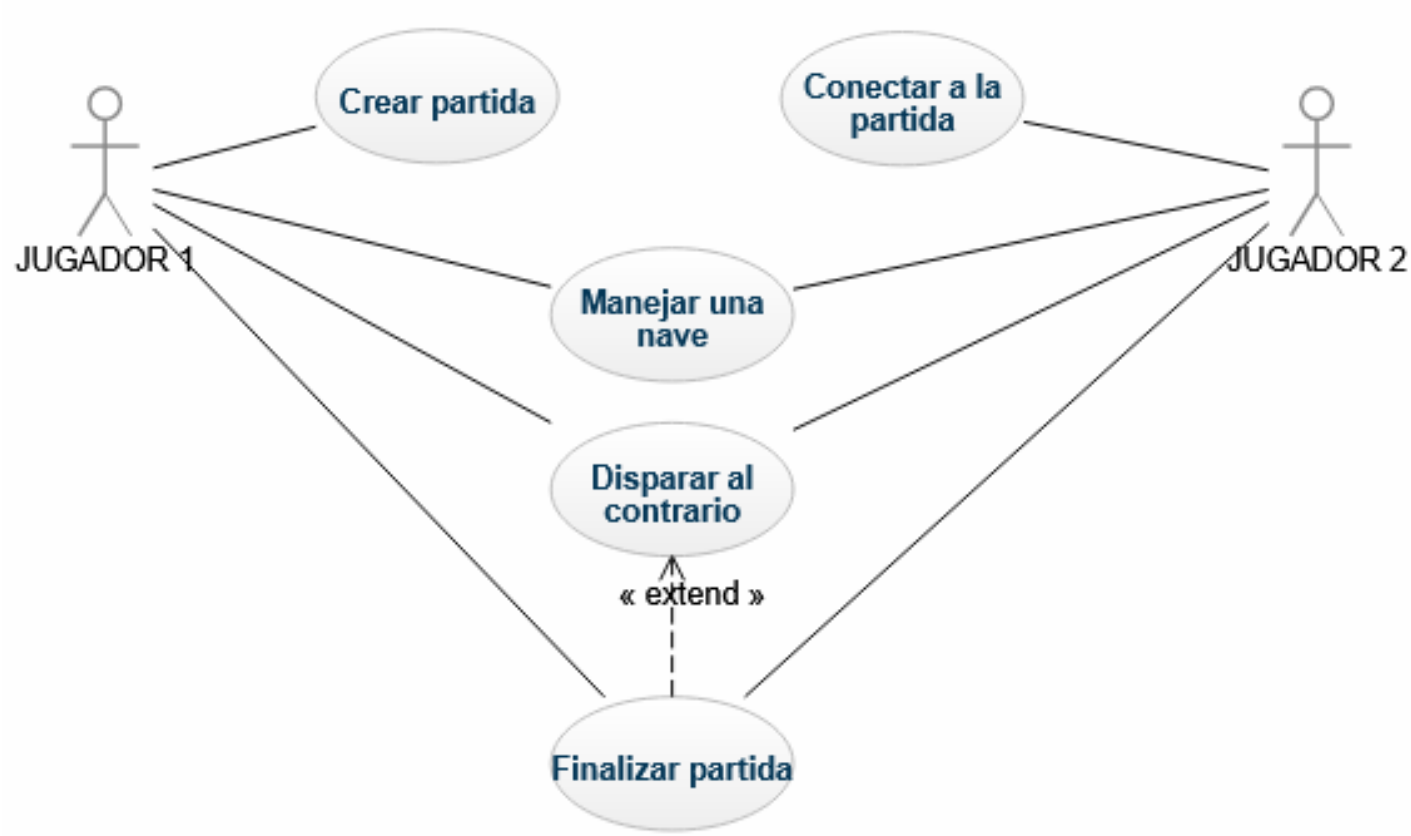
El objetivo del juego es manejar una naver y disparar al contrario.

Si uno de los dos jugadores acierta, la partida termina.

Si uno de los dos jugadores deja la partida (o se pierde la conexión), la partida termina.

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE CASOS DE USO

EJERCICIO EVALUABLE

Ejercicio 6 – reservas hotel

Un hotel permite hacer reservas de habitaciones y de ello se encarga el administrativo de reservas, y, de forma alternativa, el cliente la puede hacer también por Internet.

Siempre que se solicita la reserva de una habitación, se comprueba si la persona es cliente del hotel, en cuyo caso sólo hay que cumplimentar los datos de la reserva (fecha de entrada, fecha de salida, tipo de habitación, etc.). Si la persona no es cliente del hotel, además hay que darlo de alta e introducir sus datos personales.

Una reserva puede ser anulada en cualquier momento, pero si se realiza con menos de 24 horas de antelación, se penaliza al cliente cargándole una cantidad equivalente al 50% del coste de una noche en la habitación reservada.

Una vez el cliente llega al hotel, la persona Recepcionista comprueba su reserva y los datos del cliente y procede a registrarlo. Una vez concluida la estancia, la Recepcionista procede a facturar la estancia.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA

Los diagramas de secuencia se utilizan para formalizar la descripción de un escenario o conjunto de ellos representando **qué mensajes fluyen en el sistema así como quien los envía y quien los recibe**.

Con esto el equipo de desarrollo puede hacerse una idea de las diferentes operaciones que deben ocurrir al ejecutarse una determinada tarea y el orden en que deben realizarse.

REPRESENTACIÓN DE OBJETOS Y LÍNEA DE VIDA

En un diagrama de secuencia se dibujan las entidades (objetos y actores) que participan dentro de rectángulos que se distribuyen horizontalmente en la zona superior del diagrama. De cada rectángulo o entidad sale una línea de puntos que representa el paso del tiempo, llamada línea de vida.

Para formar el nombre de una linea de vida de un objeto se usa el nombre del objeto, que es opcional, seguido del símbolo dos puntos y a continuación el nombre de la clase a la que pertenece.

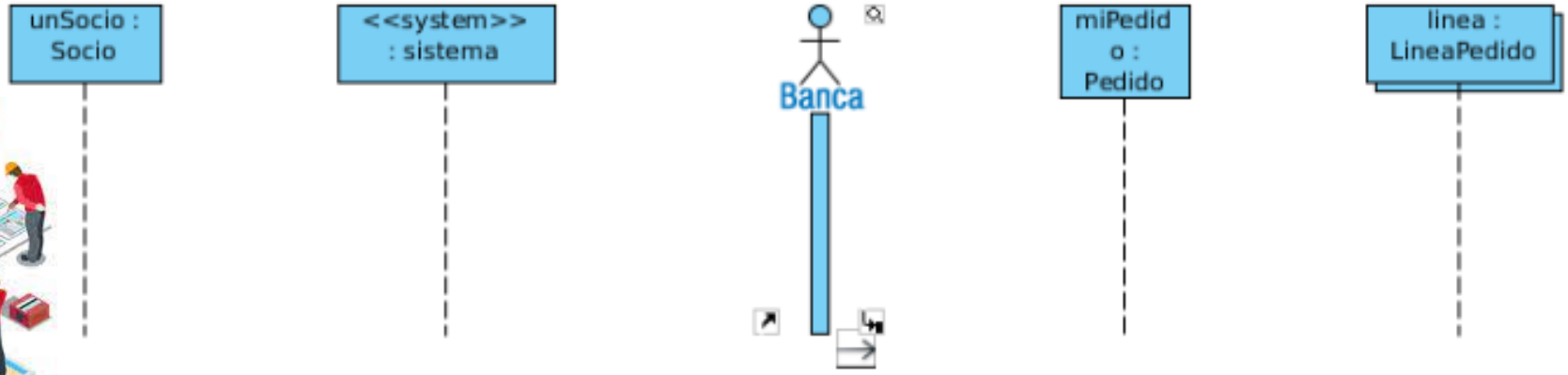
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA

REPRESENTACIÓN DE OBJETOS Y LÍNEA DE VIDA

Una línea de vida puede estar encabezada por otro tipo de instancias como el sistema o un actor que aparecerán con su propio nombre. Usaremos el sistema para representar solicitudes al mismo, como por ejemplo pulsar un botón para abrir una ventana o una llamada a una subrutina.

Cuando tenemos un objeto que puede tener varias instancias, aparece como un rectángulo sobre otro, como en es el caso de las lineas del pedido que pueden ser varias.



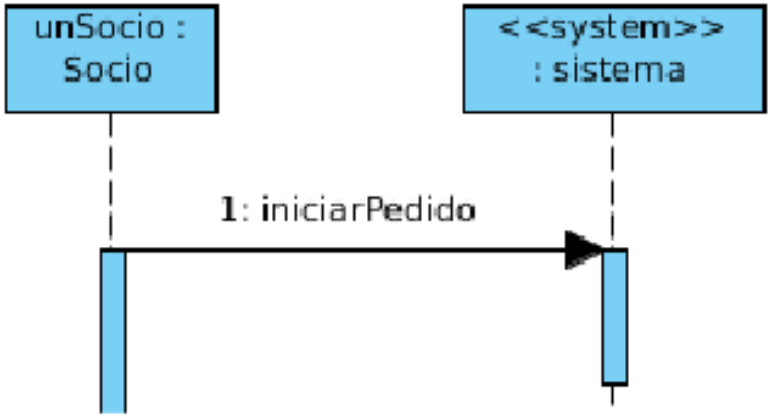
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA

INVOCACIÓN DE MÉTODOS

Los mensajes, que significan la invocación de métodos, se representan como flechas horizontales que van de una linea de vida a otra, indicando con la flecha la dirección del mensaje.

Los extremos de cada mensaje se conectan con la línea de vida de las entidades implicadas. Los mensajes se dibujan desde el objeto que envía el mensaje al que lo recibe, pudiendo ser ambos el mismo objeto, y el orden en que suceden viene determinado por su posición vertical. Un mensaje que se dibuja debajo de otro indica que se envía después, por lo que no es necesario poner números de secuencia.



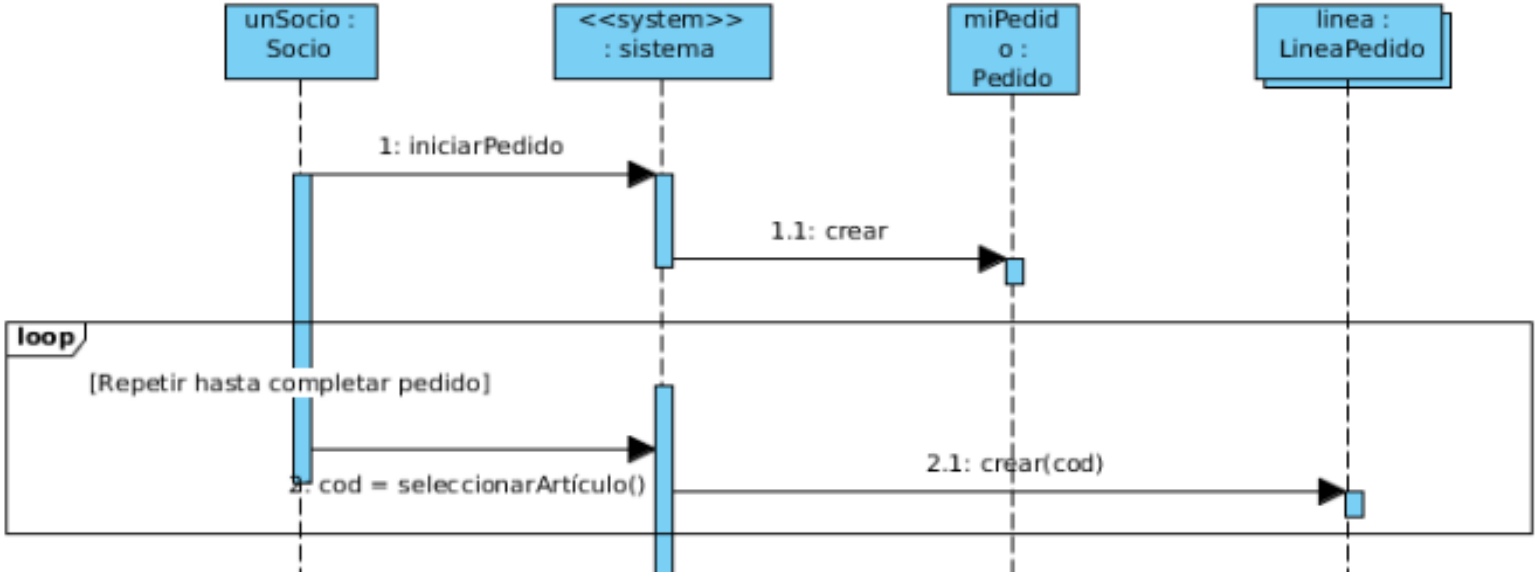
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA

ITERACIONES Y CONDICIONES

Además de presentar acciones sencillas que se ejecutan de manera secuencial, también se pueden representar algunas situaciones más complejas, como bucles, usando marcos.

Normalmente se nombra el marco con el tipo de bucle a ejecutar y la condición de parada. También se pueden representar flujos de mensajes condicionales en función de un valor determinado.



Por último, se puede completar el diagrama añadiendo etiquetas y notas en el margen izquierdo que aclaren la operación que se está realizando.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA

ELABORANDO UN DIAGRAMA DE SECUENCIA

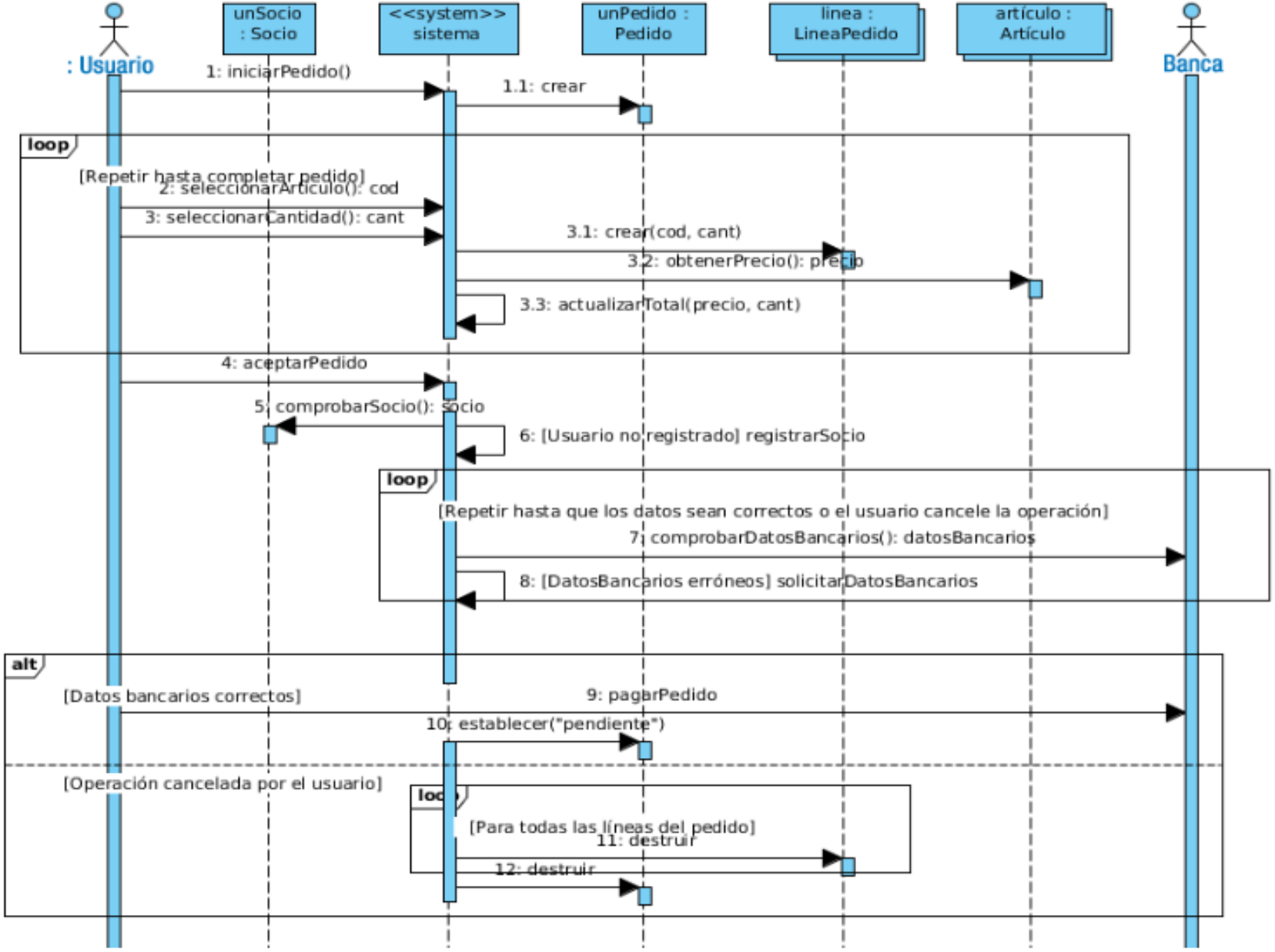
Vamos a generar el diagrama de secuencia para el caso de uso *Hacer pedido*. En él se establece la secuencia de operaciones que se llevarán a cabo entre los diferentes objetos que intervienen en el caso de uso.

En la página siguiente se presenta el diagrama ya terminado.

- Se han incluido todas las entidades (actores, objetos y sistema) que participan en el diagrama
- Se han descrito todas las operaciones, incluidos los casos especiales, como es el registro de usuarios o la gestión de los datos bancarios.
- También incluye el modelado de acciones en bucle, como es la selección de artículos y de acciones regidas por condición, como es la posibilidad de cancelar el pedido si hay problemas con la tarjeta de crédito.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

gestión de pedidos

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca

Una aplicación debe guardar la información de los pedidos que realizan sus clientes y la fecha en que se formalizan.

Los pedidos constan habitualmente de varios productos.

El coste total del pedido se calcula a partir de los precios individuales de cada producto, así como de la cantidad e impuestos asociados a cada uno.

Es importante mantener información sobre las existencias de cada producto con el fin de poder informar al cliente si habrá retrasos en la entrega del pedido.

El pedido podrá pagarse de una vez o en varios pagos.

Las formas de pago posible son

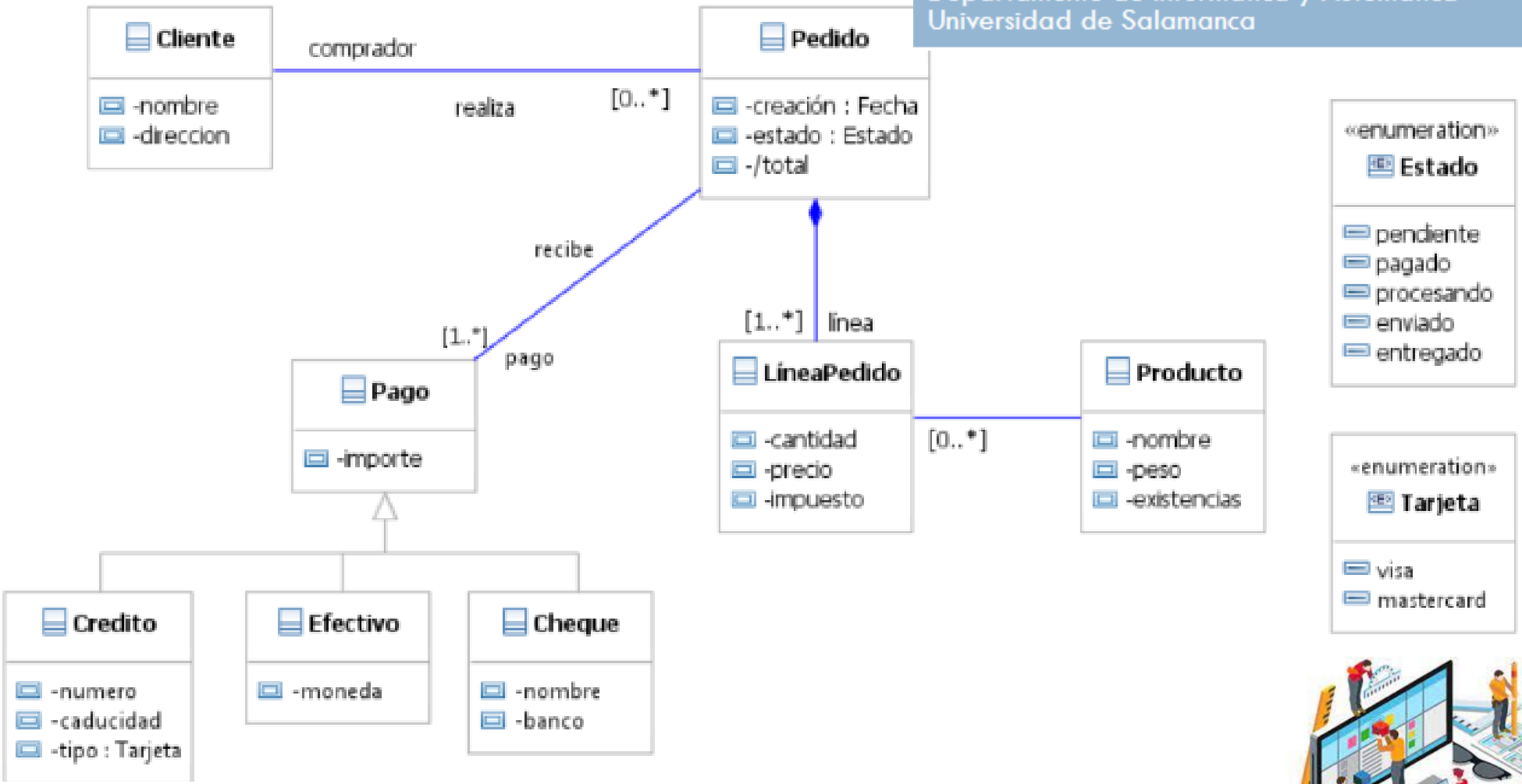
- Tarjeta de crédito (fecha de caducidad, número, VISA o MASTERCARD)
- Efectivo (moneda)
- Cheque (nombre, entidad bancaria)

El pedido podrá estar en uno de los siguientes estados: pendiente, pagado, procesando, enviado y entregado

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca

Ejercicio 1: "Realiza pedido"

Describir el diagrama de secuencia para el caso de uso "Realiza pedido", iniciado por un cliente, suponiendo que un cliente correctamente registrado ha accedido correctamente al sistema.

La secuencia de eventos que hay que modelar es la siguiente:

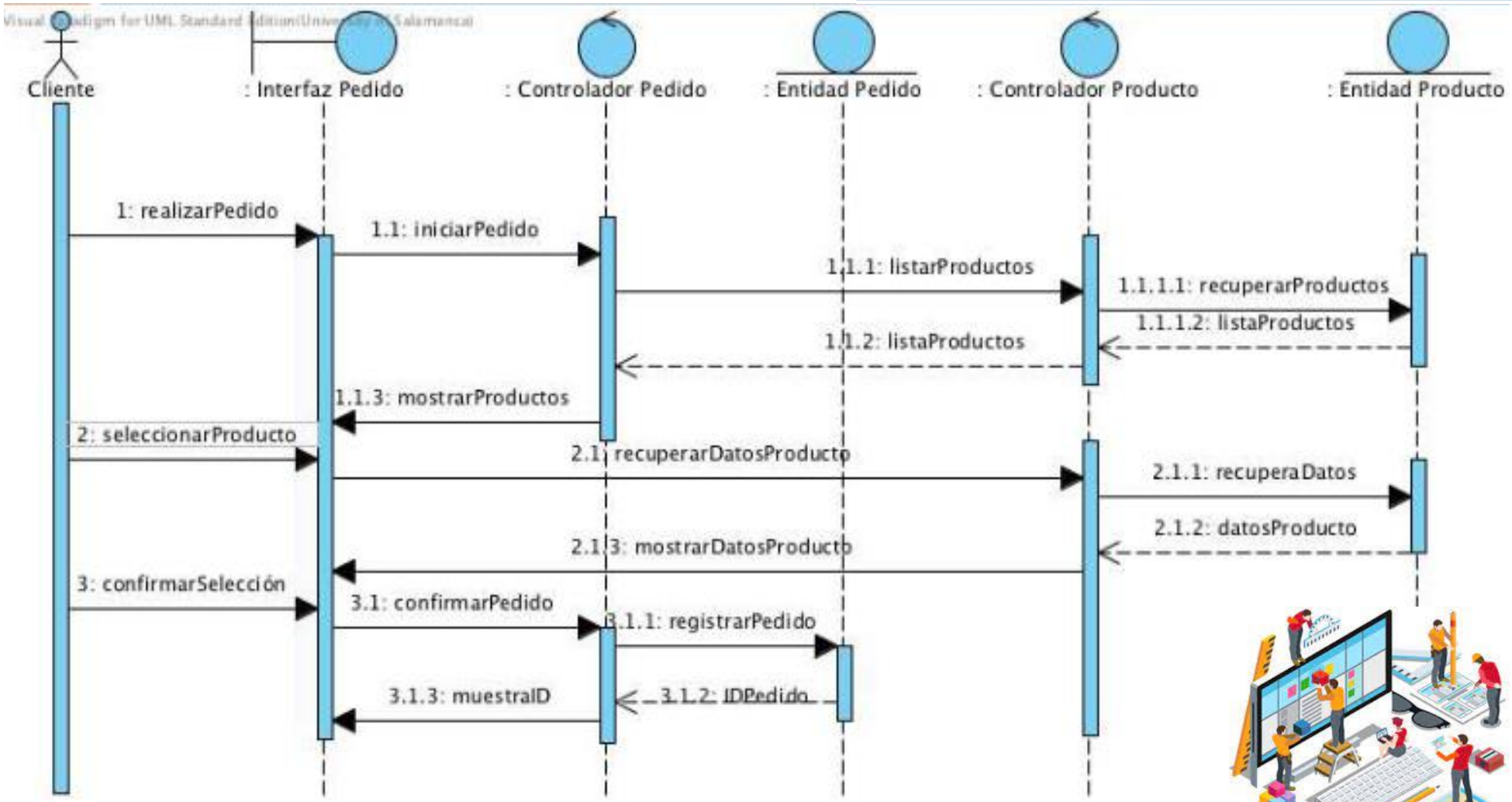
- El cliente solicita la realización de un pedido
- El sistema pedidos muestra al cliente un listado con los productos
- El cliente selecciona el producto que desea incluir en el pedido
- El sistema muestra la descripción y el precio del producto al cliente
- El cliente confirma la selección
- El sistema devuelve el id del pedido al usuario

Si el pedido no ha sido cancelado, es registrado en el sistema y confirmado al cliente

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca

Ejercicio 2: "Pagar pedido"

Describir el diagrama de secuencia para el caso de uso: "Pagar_pedido", iniciado por un cliente, suponiendo que un cliente registrado en el sistema ha accedido correctamente al sistema y va a pagar un pedido realizado previamente.

La secuencia de eventos que hay que modelar es la siguiente:

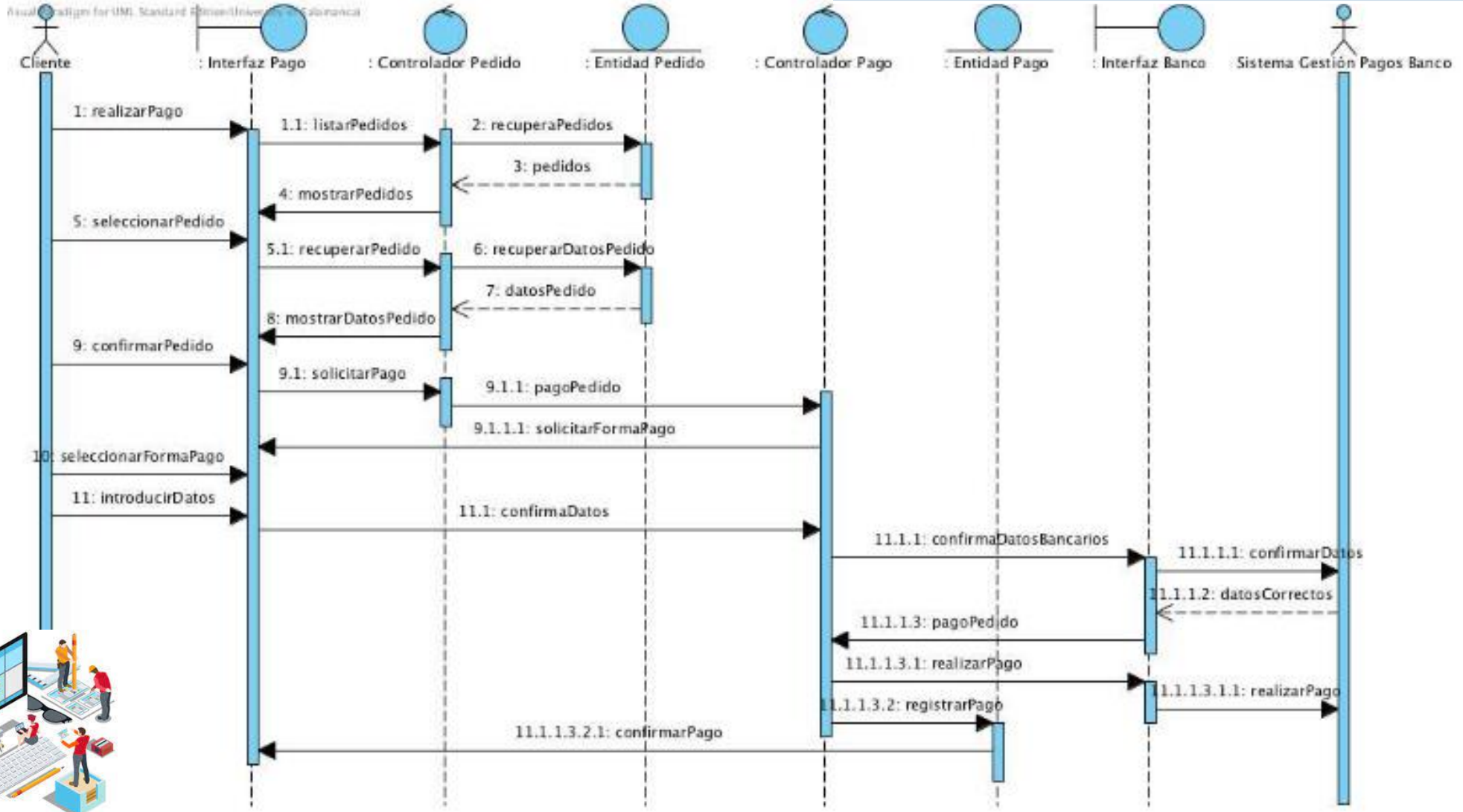
- El cliente solicita pagar un pedido
- El sistema pedidos muestra al cliente un listado con los pedidos pendientes de pago
- El cliente selecciona el pedido a pagar
- El sistema muestra el resumen del pedido
- El cliente confirma el pedido
- El sistema solicita que se seleccione una forma de pago
- El cliente selecciona una forma de pago e introduce sus datos
- El sistema de pago contacta con el banco para comprobar los datos
- El pago es confirmado se acepta el pedido y se le comunica al cliente

Si el pedido no ha sido cancelado, es registrado como pagado en el sistema

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca



DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca

Ejercicio 3: "realizar pedido con varios productos"

Realizar un diagrama de secuencia en la que se realice la misma secuencia de Realizar pedido pero pudiendo seleccionar varios productos.

La secuencia de eventos que hay que modelar es la siguiente:

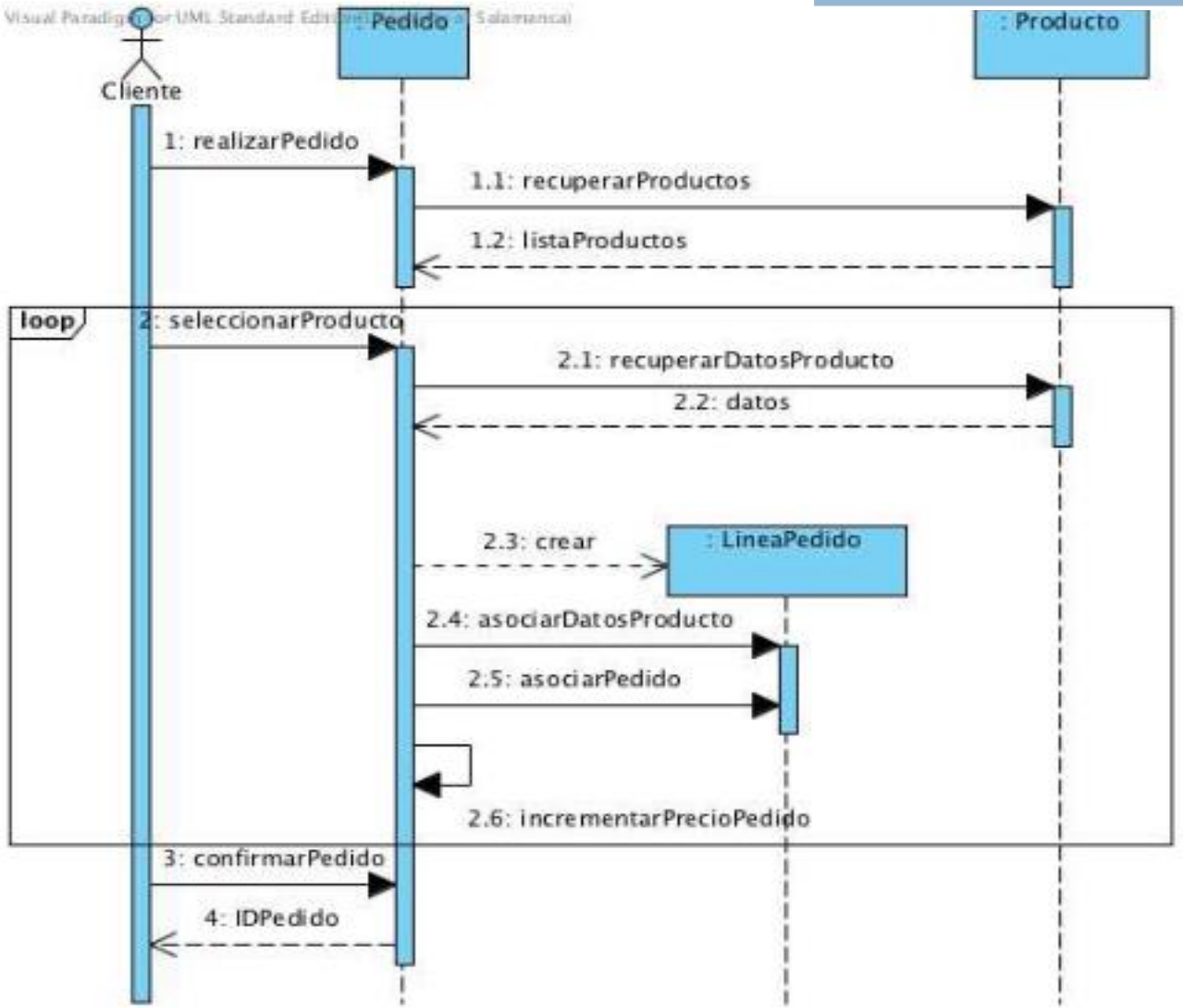
- El cliente solicita la realización de un pedido.
- El sistema pedidos muestra al cliente un listado con los productos
- El cliente selecciona los productos que desea incluir en el pedido
- Por cada producto
    - El sistema crea una línea de pedido sobre la que se aporta descripción, precio y cantidad
    - Dicha línea de pedido se asocia al pedido
    - El sistema suma el precio del producto en el total del pedido
- El cliente confirma la selección
- El sistema devuelve el id del pedido al usuario

Si el pedido no ha sido cancelado, es registrado en el sistema y confirmado al cliente.

DIAGRAMAS DE COMPORTAMIENTO

EJERCICIOS DE DIAGRAMAS DE SECUENCIA

(CC) García, F.J.; Moreno, M.; Bravo, S.; Conde, M.A.
Departamento de Informática y Automática
Universidad de Salamanca



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

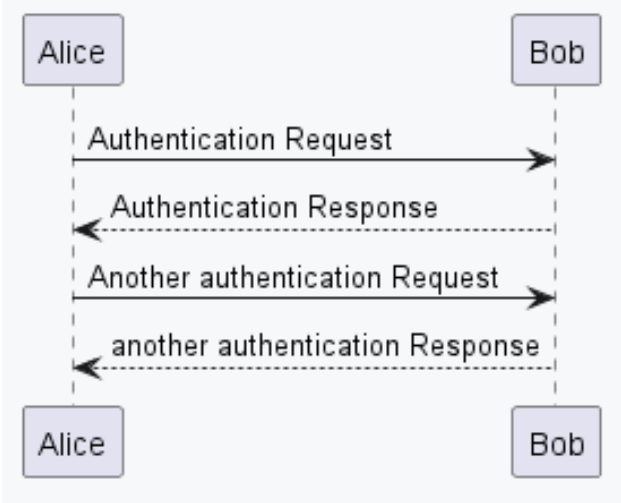
Ejemplo básico

La secuencia -> es usada para dibujar un mensaje entre dos participantes. Los participantes tienen que ser declarados explícitamente.

Para definir una flecha punteada, se debe usar -->

También se puede usar <- y <--. No provoca cambios en el dibujo, pero puede mejorar la legibilidad. Tenga en cuenta que esto sólo es posible en diagramas de secuencia, las reglas son diferentes para otros diagramas.

```
@startuml
Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

Alice -> Bob: Another authentication Request
Alice <-- Bob: another authentication Response
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

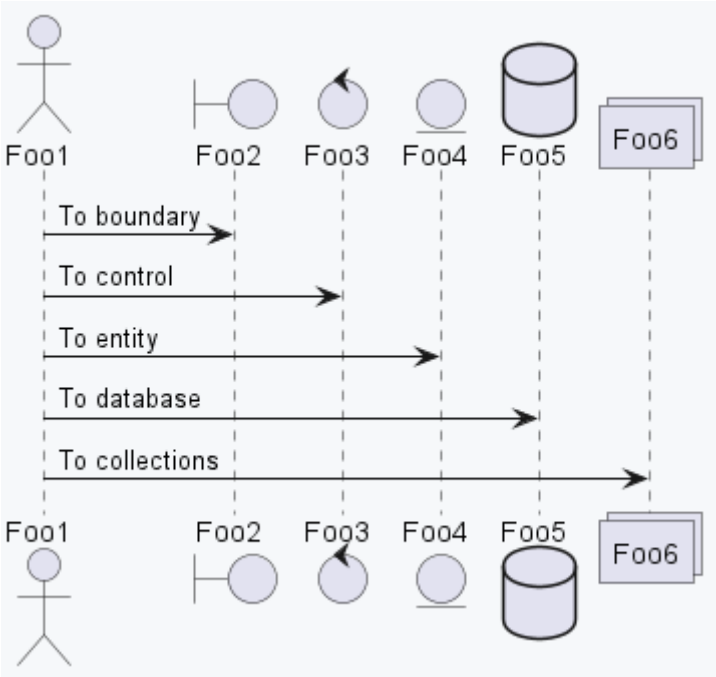
Declarando participantes

Es posible cambiar el orden de los participantes usando la palabra reservada `participant`.

También es posible el uso de otras palabras reservadas para declarar un participante:

- `actor`
- `boundary`
- `control`
- `entity`
- `database`
- `collections`

```
@startuml
actor Foo1
boundary Foo2
control Foo3
entity Foo4
database Foo5
collections Foo6
Foo1 -> Foo2 : To boundary
Foo1 -> Foo3 : To control
Foo1 -> Foo4 : To entity
Foo1 -> Foo5 : To database
Foo1 -> Foo6 : To collections

@enduml
```



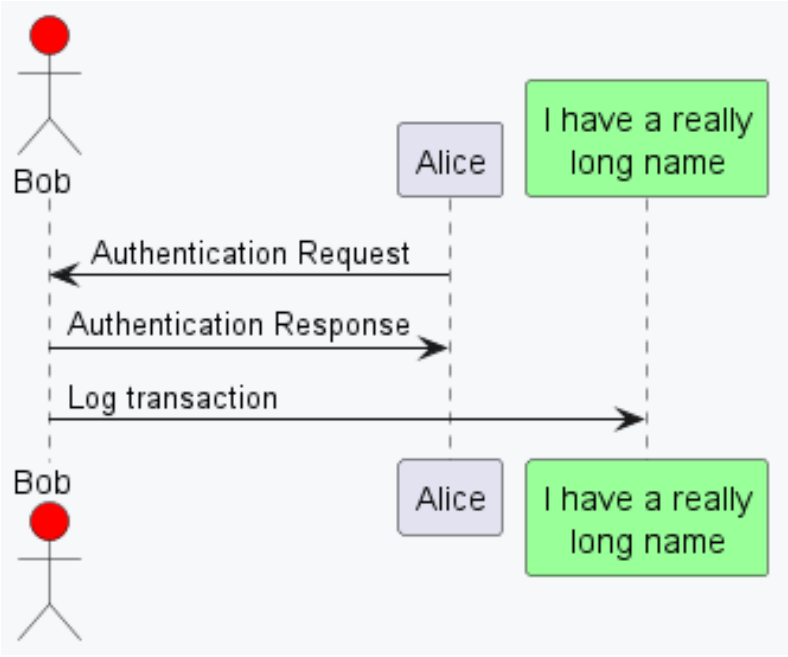
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Se puede renombrar un participante usando la palabra reservada `as` .

También es posible cambiar el color de fondo de los actores o participantes.

```
@startuml
actor Bob #red
' The only difference between actor
'and participant is the drawing
participant Alice
participant "I have a really\nlong name" as L
#99FF99
/' You can also declare:
   participant L as "I have a really\nlong name"
#99FF99
  '/

Alice->Bob: Authentication Request
Bob->Alice: Authentication Response
Bob->L: Log transaction
@enduml
```

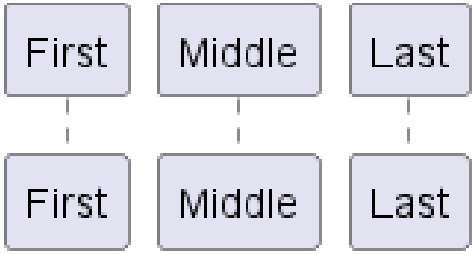


DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

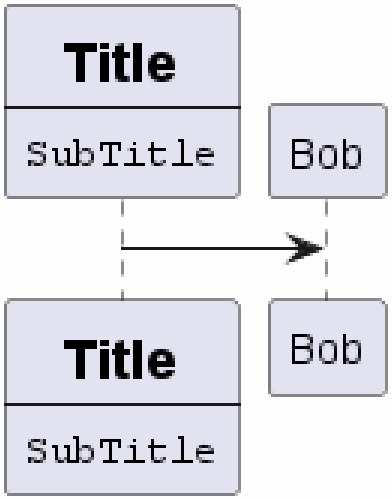
Puede utilizar la palabra clave *order* para personalizar el orden de impresión del participante.

```
@startuml
participant Last order 30
participant Middle order 20
participant First order 10
@enduml
```



Declaración de Participantes en múltiples líneas:

```
@startuml
participant Participant [
    =Title
    ----
    ""SubTitle""
]

participant Bob

Participant -> Bob
@enduml
```



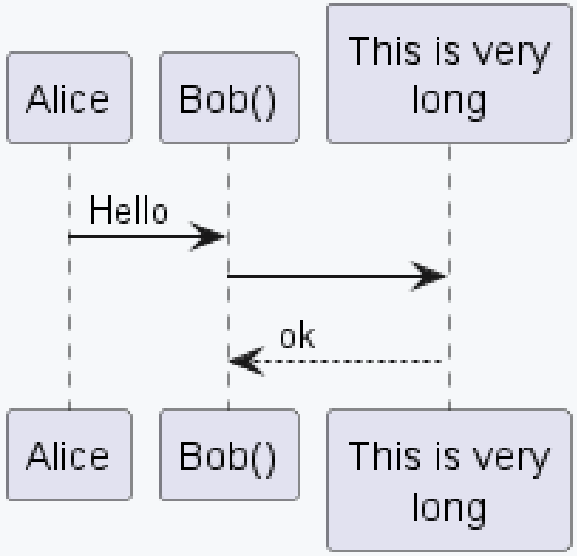
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Sin usar letras en participantes

Puedes usar comillas para definir participantes. Y puedes usar la palabra reservada as para asignar un alias a esos participantes.

```
@startuml
Alice -> "Bob()" : Hello
"Bob()" -> "This is very\nlong" as Long
' You can also declare:
' "Bob()" -> Long as "This is very\nlong"
Long --> "Bob()" : ok
@enduml
```



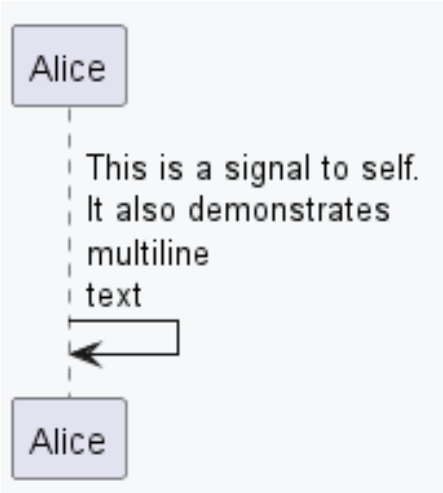
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

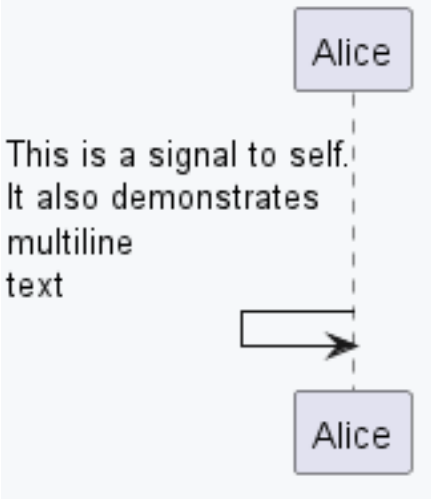
Mensaje a sí mismo

Un participante puede enviarse un mensaje a sí mismo. También es posible tener varias líneas utilizando \n.

```
@startuml
Alice -> Alice: This is a signal to self.\nIt also demonstrates\nmultiline \ntext
@enduml
```




```
@startuml
Alice <- Alice: This is a signal to self.\nIt also demonstrates\nmultiline \ntext
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

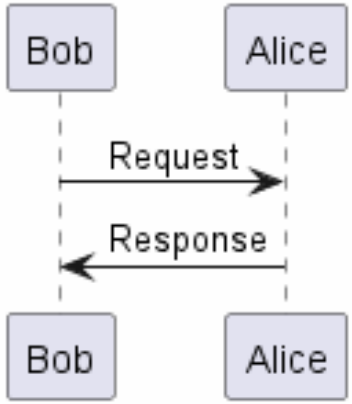
DIAGRAMAS DE SECUENCIA con PlantUML

Alineación del texto

La alineación del texto en las flechas puede establecerse en left, right o center utilizando skinparam sequenceMessageAlign.

También puede utilizar direction o reverseDirection para alinear el texto dependiendo de la dirección de la flecha. Encontrará más detalles y ejemplos en la página skinparam.

```
@startuml
skinparam sequenceMessageAlign right
Bob -> Alice : Request
Alice -> Bob : Response
@enduml
```



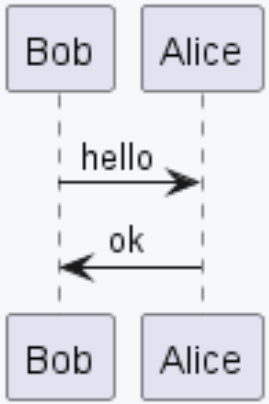
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Texto del mensaje de respuesta<zh3>debajo de la flecha

Puede poner el texto del mensaje de respuesta debajo de la flecha, con el comando skinparam responseMessageBelowArrow true.

```
@startuml
skinparam responseMessageBelowArrow true
Bob -> Alice : hello
Alice -> Bob : ok
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

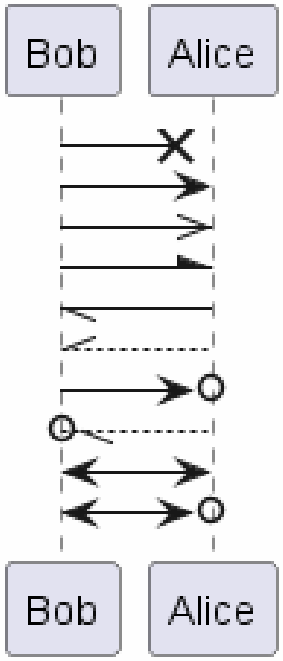
DIAGRAMAS DE SECUENCIA con PlantUML

Cambiar estilo de la flecha

Puede cambiar el estilo de la flecha de diferentes formas:

- añade una `x` al final para indicar un mensaje perdido
- utilice `\` o `/` en lugar de `<` o `>` para tener solo la parte inferior o superior de la flecha
- repite la cabeza de la flecha (por ejemplo, `>>` o `//`) para tener un trazo más fino
- Utilice `--` en lugar de `-` para obtener una flecha punteada.
- añade una "o" al final de la cabeza de una flecha
- utilice flechas bidireccionales `<->`

```
@startuml
Bob ->x Alice
Bob -> Alice
Bob ->> Alice
Bob -\ Alice
Bob \\- Alice
Bob //-- Alice

Bob ->o Alice
Bob o\\-- Alice

Bob <-> Alice
Bob <->o Alice
@enduml
```



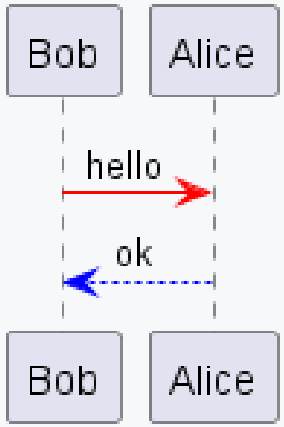
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Cambiar el color de la flecha

Puede cambiar el color de flechas individuales usando la siguiente notación:

```
@startuml
Bob -[#red]> Alice : hello
Alice -[#0000FF]->Bob : ok
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

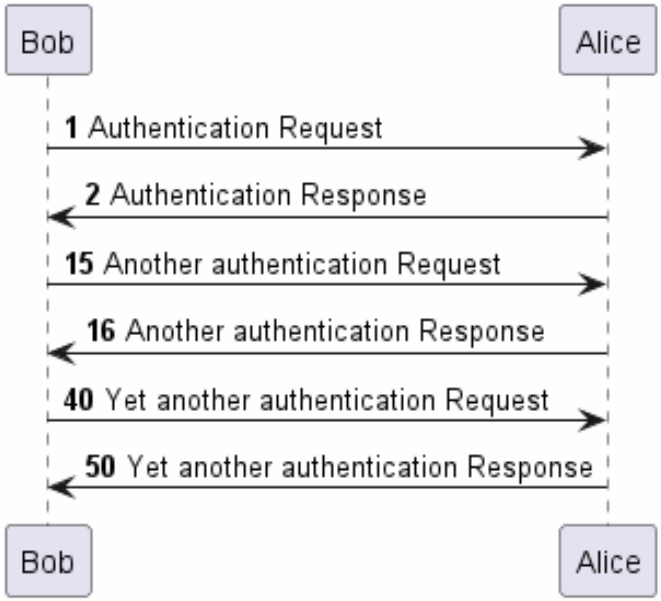
Numeración de la secuencia de mensajes

La palabra clave `autonumber` es usada para añadir automáticamente números a los mensajes.

```
@startuml
autonumber
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response
@enduml
```

Puedes especificar un número de comienzo con autonumber //número inicial// , y también un incremento con autonumber //número inicial// //incremento//.

```
@startuml
autonumber
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response

autonumber 15
Bob -> Alice : Another authentication Request
Bob <- Alice : Another authentication Response

autonumber 40 10
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

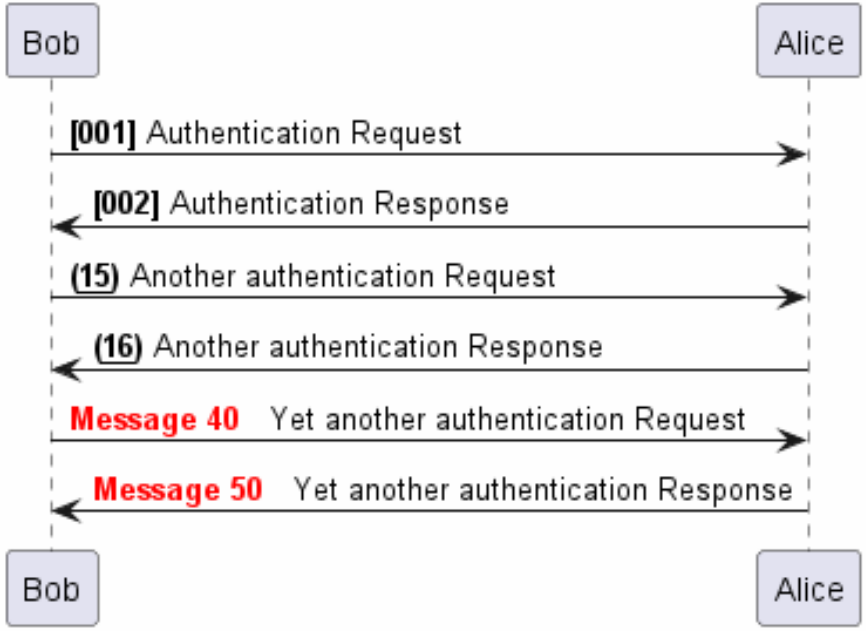
DIAGRAMAS DE SECUENCIA con PlantUML

Puedes especificar un formato para su número usándolo entre comillas dobles.

El formateo se hace mediante la calse Java DecimalFormat (0 denota un dígito, # denota un digito y cero si está ausente).

Puedes usar alguna etiqueta HTML en el formato.

```
@startuml
autonumber "<b>[000]"
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response

autonumber 15 "<b>(<u>##</u>)"
Bob -> Alice : Another authentication Request
Bob <- Alice : Another authentication Response

autonumber 40 10 "<font color=red><b>Message 0  "
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response

@enduml
```

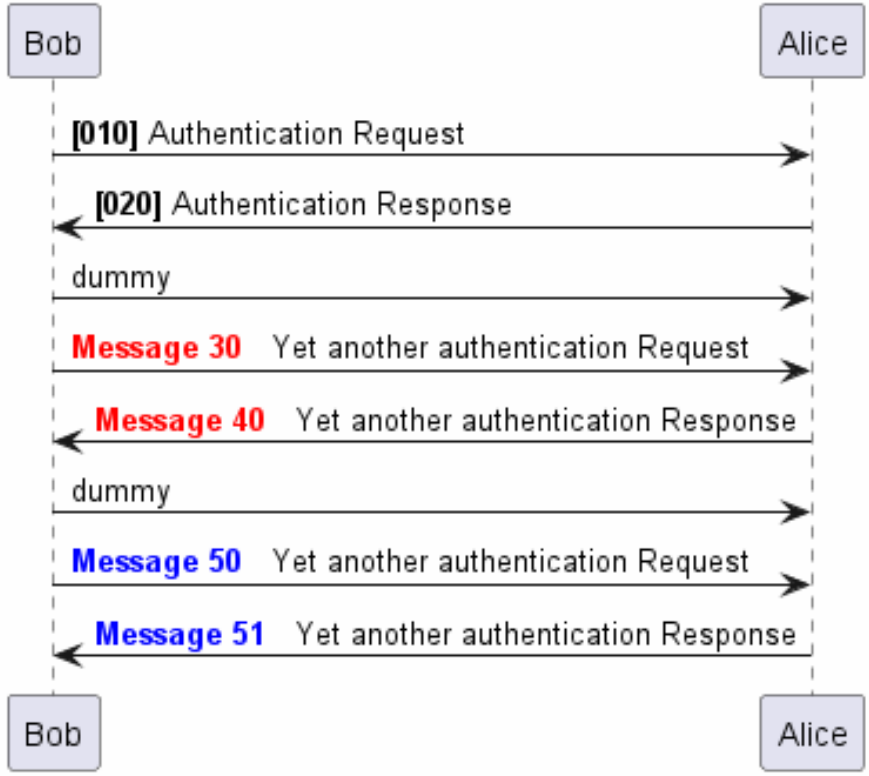


DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

También puedes usar autonumber stop y autonumber resume //increment// //format// para pausar y continuar la numeración automática, respectivamente.

```
@startuml
autonumber 10 10 "<b>[000]"
Bob -> Alice : Authentication Request
Bob <- Alice : Authentication Response

autonumber stop
Bob -> Alice : dummy

autonumber resume "<font color=red><b>Message 0  "
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response

autonumber stop
Bob -> Alice : dummy

autonumber resume 1 "<font color=blue><b>Message 0  "
Bob -> Alice : Yet another authentication Request
Bob <- Alice : Yet another authentication Response
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

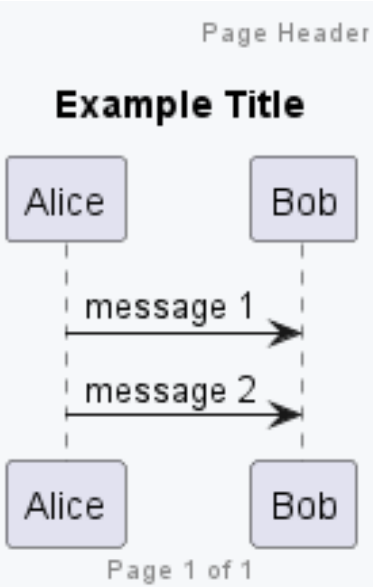
DIAGRAMAS DE SECUENCIA con PlantUML

Título de página, encabezado y pie de página

La palabra clave title se utiliza para añadir un título a la página.

Las páginas pueden mostrar encabezados y pies de página utilizando header y footer.

```
@startuml

header Page Header
footer Page %page% of %lastpage%

title Example Title

Alice -> Bob : message 1
Alice -> Bob : message 2

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

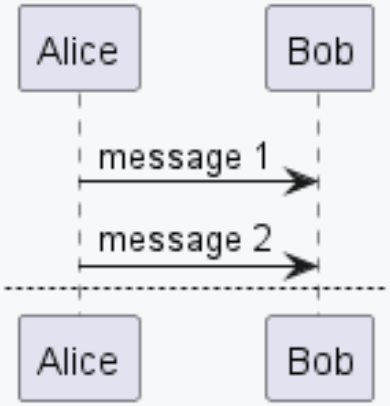
Dividiendo diagramas

La palabra reservada `newpage` es empleada para dividir un diagrama en varias imágenes.

Puedes colocar un título para la página nueva justo después de la palabra reservada `newpage` .

Esto es bastante práctico con *Word* para devolver diagramas grandes en varias páginas.

```
@startuml

Alice -> Bob : message 1
Alice -> Bob : message 2

newpage

Alice -> Bob : message 3
Alice -> Bob : message 4

newpage A title for the\nlast page

Alice -> Bob : message 5
Alice -> Bob : message 6
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Agrupando mensajes

Es posible agrupar mensajes usando las siguientes palabras reservadas:

- `alt/else`
- `opt`
- `loop`
- `par`
- `break`
- `critical`
- `group`, seguida de un texto para mostrar

Es posible añadir un texto que será mostrado en el encabezado (excepto para `group`).

La palabra reservada `end` es usada para cerrar el grupo.

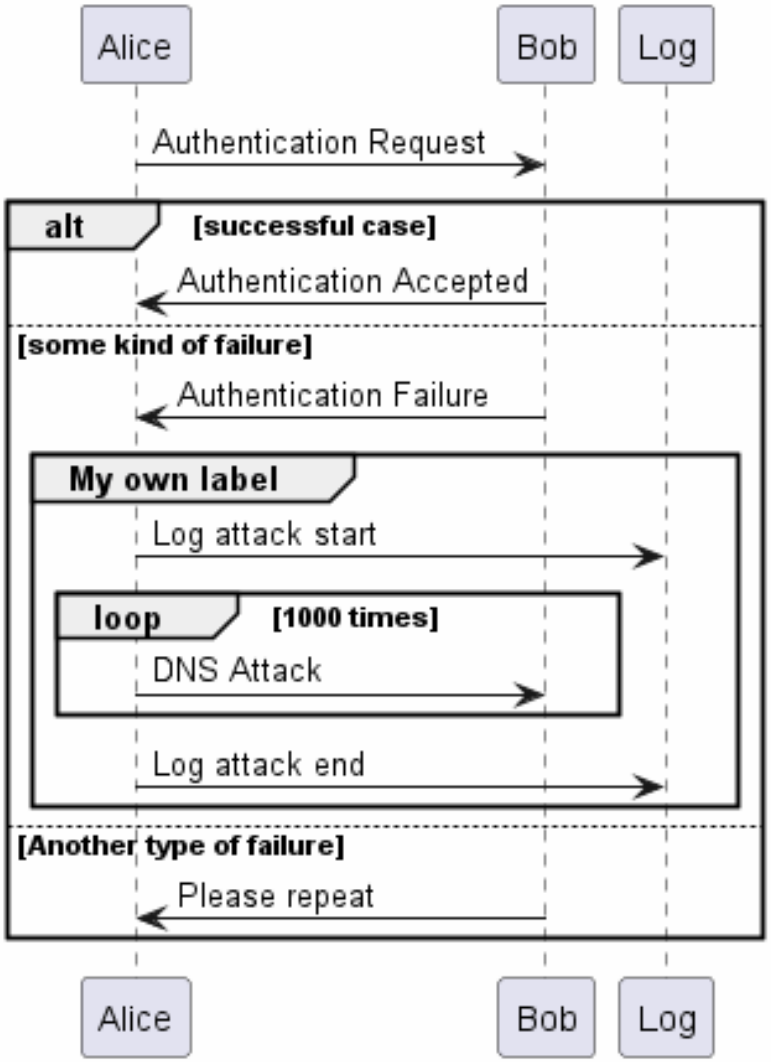
Tenga en cuenta que es posible anidar grupos.

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Agrupando mensajes

```
@startuml
Alice -> Bob: Authentication Request

alt successful case

    Bob -> Alice: Authentication Accepted

else some kind of failure

    Bob -> Alice: Authentication Failure
    group My own label
    Alice -> Log : Log attack start
        loop 1000 times
            Alice -> Bob: DNS Attack
        end
    Alice -> Log : Log attack end
    end

else Another type of failure

   Bob -> Alice: Please repeat

end
@enduml
```



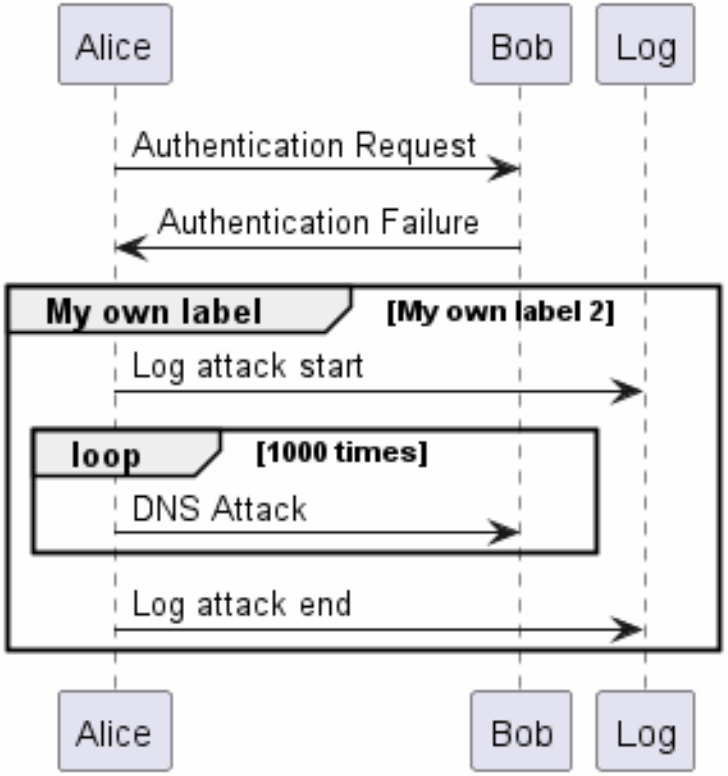
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Etiqueta secundaria de grupo

Para group, es posible añadir, entre[y], un texto o etiqueta secundaria que se mostrará en la cabecera.

```
@startuml
Alice -> Bob: Authentication Request
Bob -> Alice: Authentication Failure
group My own label [My own label 2]
    Alice -> Log : Log attack start
    loop 1000 times
        Alice -> Bob: DNS Attack
    end
    Alice -> Log : Log attack end
end
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

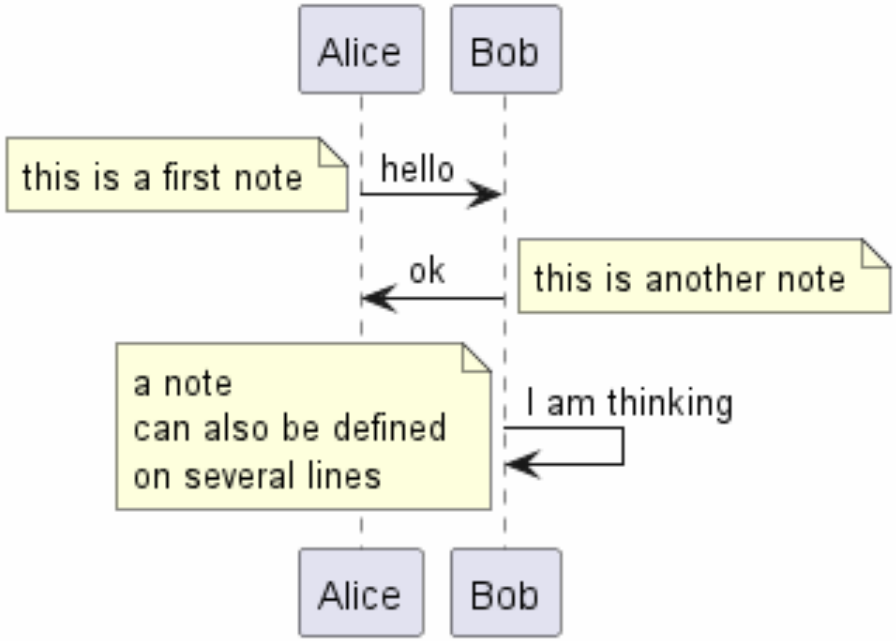
DIAGRAMAS DE SECUENCIA con PlantUML

Notas en mensajes

Es posible colocar notas en mensajes usando las palabras reservadas note left o note rightinmediatamente después del mensaje.

Puedes tener una nota multi-líneas usando la palabra reservada end note .

```
@startuml
Alice->Bob : hello
note left: this is a first note

Bob->Alice : ok
note right: this is another note

Bob->Bob : I am thinking
note left
a note
can also be defined
on several lines
end note
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

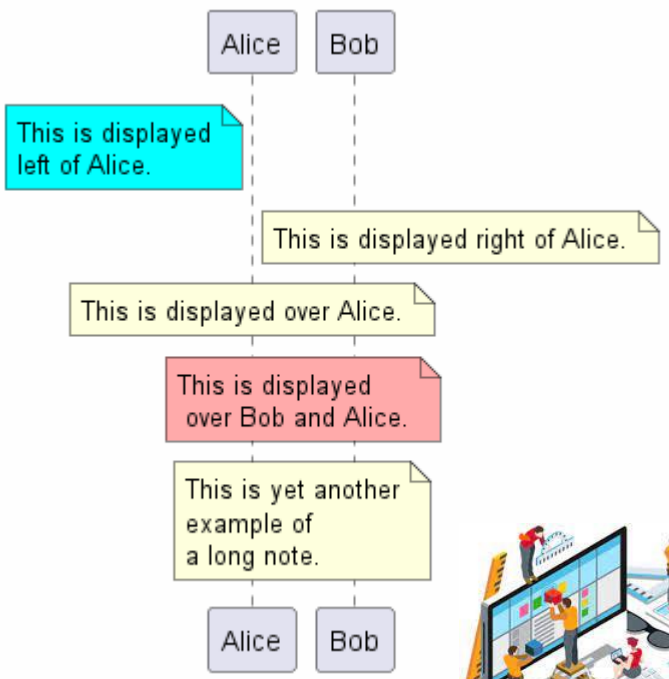
DIAGRAMAS DE SECUENCIA con PlantUML

También es posible colocar notas relativas al participante con las palabras reservadas <code>note left of</code> , note right of o note over .

Es posible resaltar una nota cambiando su color de fondo.

También puedes tener una nota multi-líneas usando la palabra reservada end note .

```
@startuml
participant Alice
participant Bob
note left of Alice #aqua
This is displayed
left of Alice.
end note

note right of Alice: This is displayed right of Alice.

note over Alice: This is displayed over Alice.

note over Alice, Bob #FFAAAA: This is displayed\n over Bob and Alice.

note over Bob, Alice
This is yet another
example of
a long note.
end note
@enduml
```



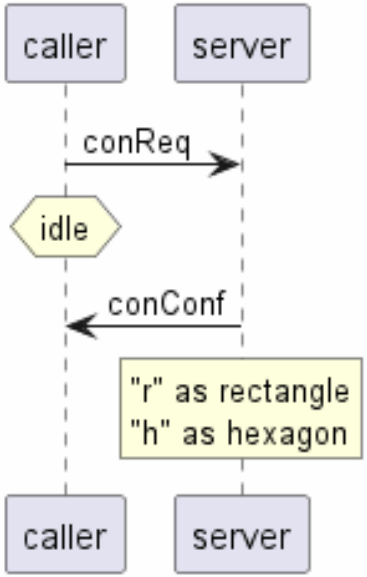
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Cambiando el aspecto de las notas

Puedes usar las palabras reservadas `hnote` y `rnote` para cambiar el aspecto de las notas.

```
@startuml
caller -> server : conReq
hnote over caller : idle
caller <- server : conConf
rnote over server
 "r" as rectangle
 "h" as hexagon
endrnote
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

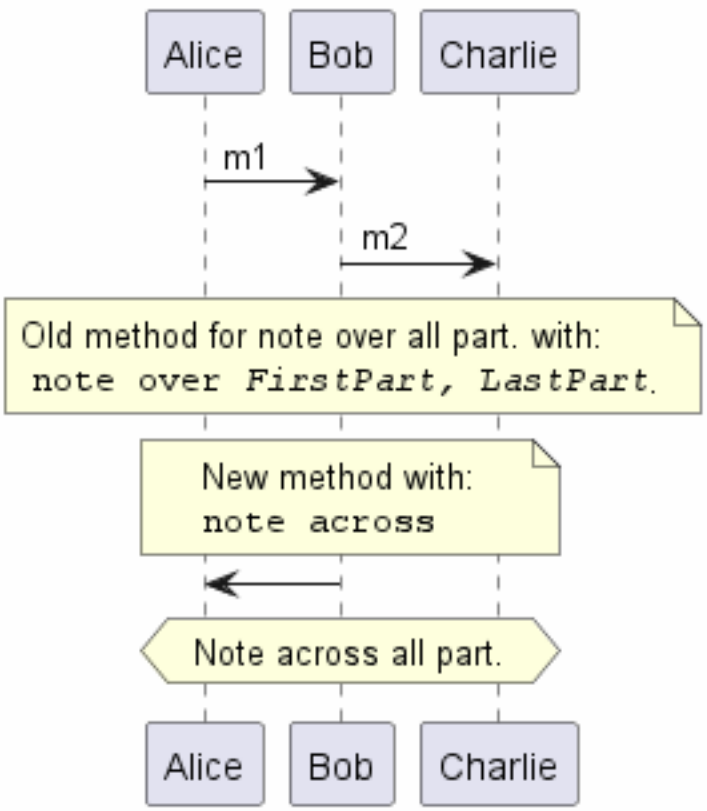
DIAGRAMAS DE SECUENCIA con PlantUML

Nota sobre todos los participantes [a través de]

Puedes hacer directamente una nota sobre todos los participantes, con la sintaxis:

note across: note_description

```
@startuml
Alice->Bob:m1
Bob->Charlie:m2
note over Alice, Charlie: Old method for note over
all part. with:\n ""note over //FirstPart, LastPart//"".
note across: New method with:\n""note across""
Bob->Alice
hnote across:Note across all part.
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

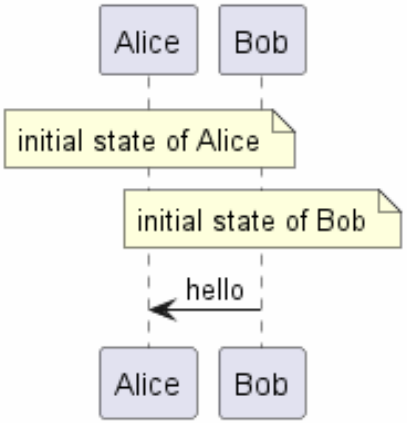
DIAGRAMAS DE SECUENCIA con PlantUML

Varias notas alineadas al mismo nivel [/]

Puedes hacer que varias notas estén alineadas al mismo nivel, con la sintaxis /:

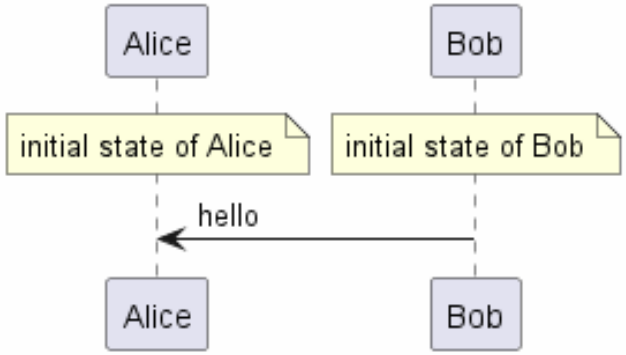
sin / (por defecto, las notas no están alineadas)

```
@startuml
note over Alice : initial state of Alice
note over Bob : initial state of Bob
Bob -> Alice : hello
@enduml
```



con / (las notas están alineadas)

```
@startuml
note over Alice : initial state of Alice
/ note over Bob : initial state of Bob
Bob -> Alice : hello
@enduml
```



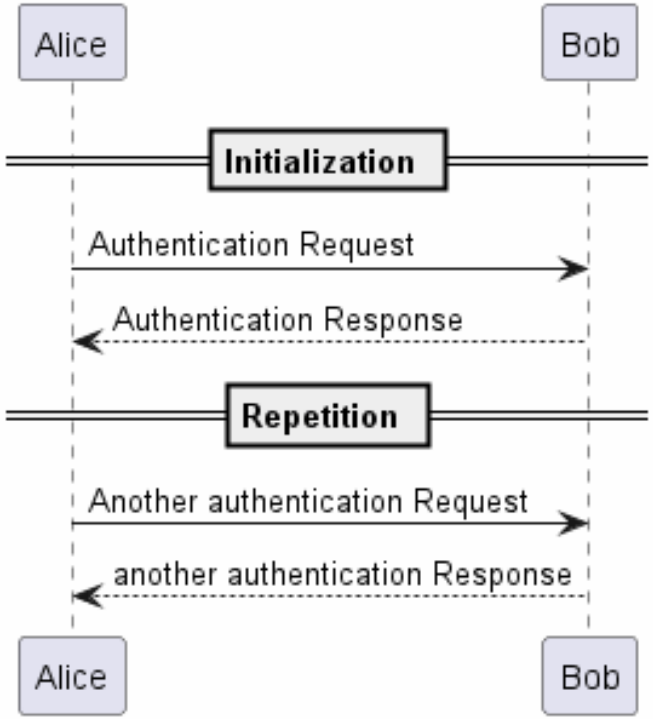
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Divisor

Si quieres, puedes dividir un diagrama usando el separador `==` para separar su diagrama en pasos lógicos.

```
@startuml

== Initialization ==

Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

== Repetition ==

Alice -> Bob: Another authentication Request
Alice <-- Bob: another authentication Response

@enduml
```



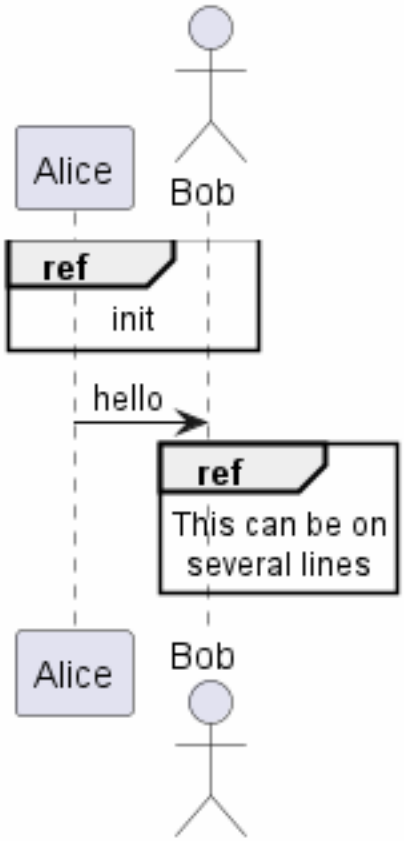
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Referencia

Puedes referenciar en un diagrama utilizando la palabra clave `ref over`.

```
@startuml
participant Alice
actor Bob

ref over Alice, Bob : init

Alice -> Bob : hello

ref over Bob
  This can be on
  several lines
end ref
@enduml
```



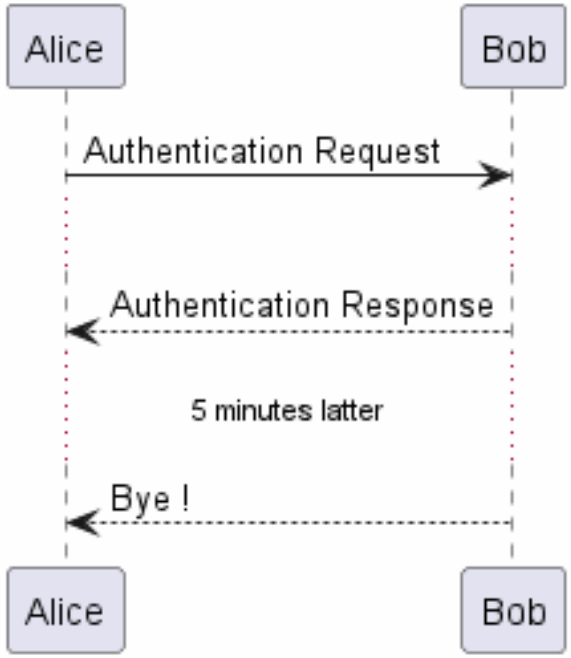
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Retardo

Puedes usar `...` para indicar un retardo en el diagrama. Y también es posible colocar un mensaje con ese retardo

```
@startuml

Alice -> Bob: Authentication Request
...
Bob --> Alice: Authentication Response
...5 minutes latter...
Bob --> Alice: Bye !

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

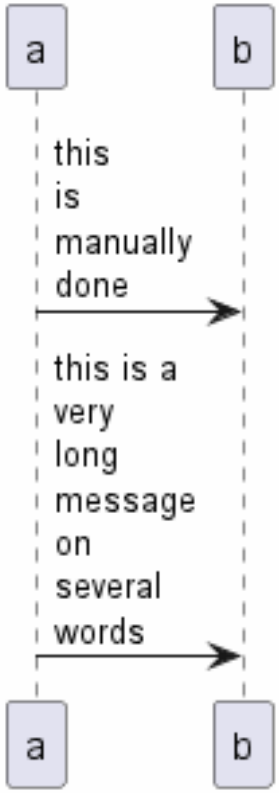
DIAGRAMAS DE SECUENCIA con PlantUML

Ajuste del texto

Para interrumpir mensajes largos, puede añadir manualmente \n en el texto.

Otra opción es utilizar el ajuste maxMessageSize:

```
@startuml
skinparam maxMessageSize 50
participant a
participant b
a -> b :this\nis\nmanually\ndone
a -> b :this is a very long message on several words
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

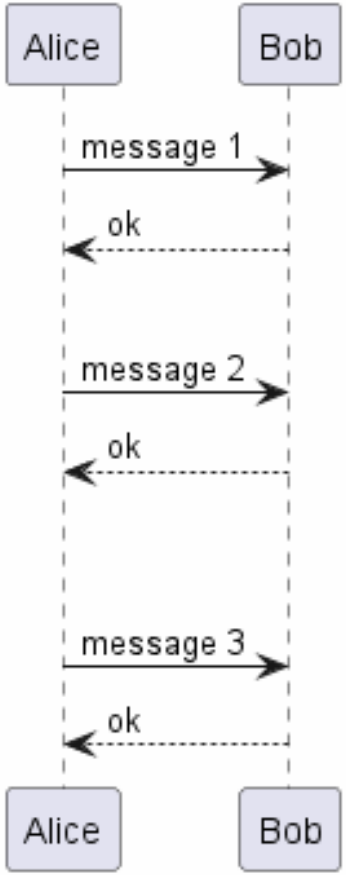
DIAGRAMAS DE SECUENCIA con PlantUML

Espaciado

Puedes usar `|||` para indicar espaciado en el diagrama.

También es posible especificar un número de píxel para ser usado.

```
@startuml

Alice -> Bob: message 1
Bob --> Alice: ok
|||
Alice -> Bob: message 2
Bob --> Alice: ok
||45||
Alice -> Bob: message 3
Bob --> Alice: ok

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Activación y Destrucción de la Línea de vida

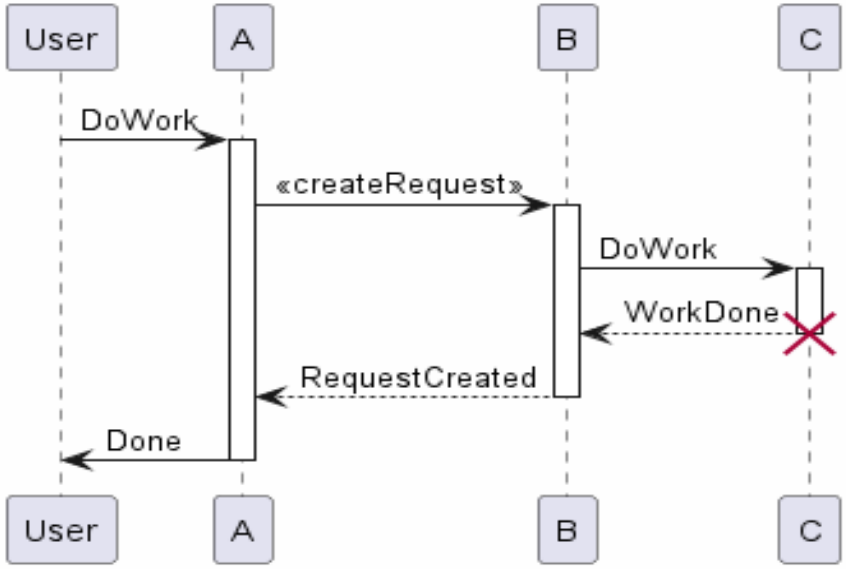
`activate` y `deactivate` son usados para denotar la activación de un participante.

Una vez que un participante es activado, su línea de vida aparece.

`activate` y `deactivate` aplica en el mensaje anterior.

`destroy` denota el final de la línea de vida de un participante.

```
@startuml
participant User

User -> A: DoWork
activate A

A -> B: << createRequest >>
activate B

B -> C: DoWork
activate C
C --> B: WorkDone
destroy C

B --> A: RequestCreated
deactivate B

A -> User: Done
deactivate A

@enduml
```

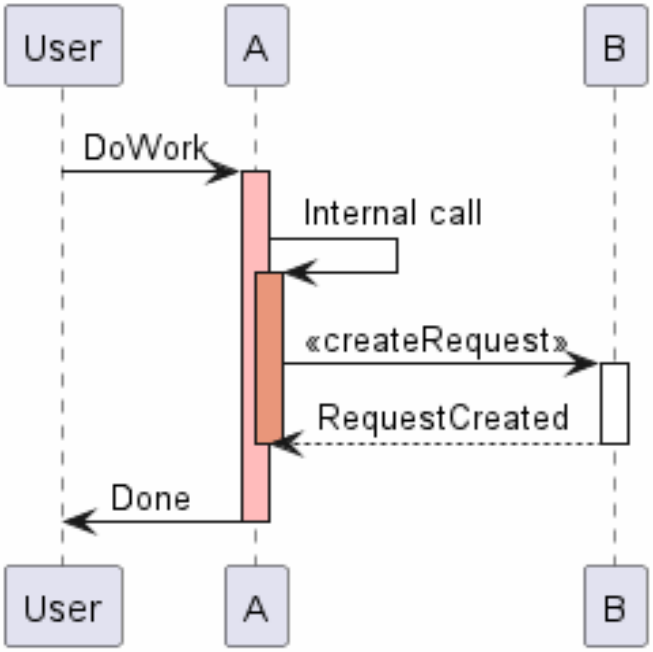


DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Puede usarse anidamiento de líneas de vida, y es posible agregar un color a dicha línea de vida.

```
@startuml
participant User

User -> A: DoWork
activate A #FFBBBB

A -> A: Internal call
activate A #DarkSalmon

A -> B: << createRequest >>
activate B

B --> A: RequestCreated
deactivate B
deactivate A
A -> User: Done
deactivate A

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

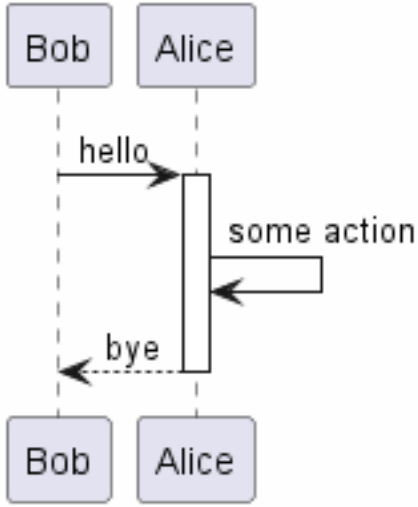
Return

El comando return genera un mensaje de retorno con una etiqueta de texto opcional.

El punto de retorno es el que causó la activación más reciente de la línea de vida.

La sintaxis es return label donde label si se proporciona es cualquier cadena aceptable para mensajes convencionales.

```
@startuml
Bob -> Alice : hello
activate Alice
Alice -> Alice : some action
return bye
@enduml
```



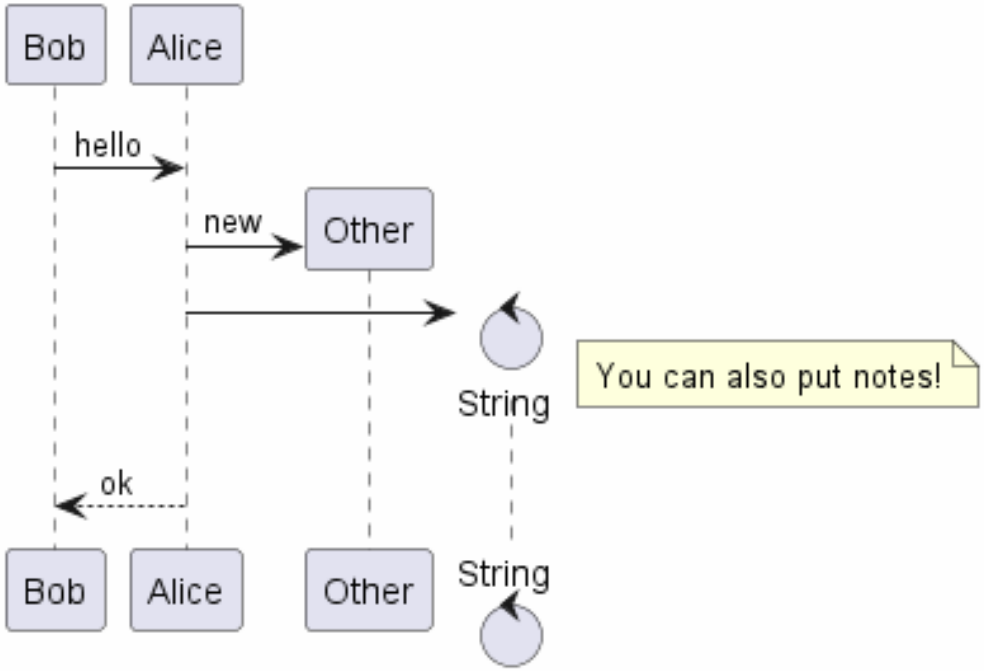
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Creación de participante

Puedes usar la palabra reservada create justo antes de la primera recepción de un mensaje para recalcar el hecho de que ese mensaje se encuentra creando ese nuevo objeto.

```
@startuml
Bob -> Alice : hello

create Other
Alice -> Other : new

create control String
Alice -> String
note right : You can also put notes!

Alice --> Bob : ok

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Sintaxis abreviada para la activación, desactivación y creación

Inmediatamente después de especificar el participante de destino, se puede utilizar la siguiente sintaxis:

++ Activar el objetivo * (opcionalmente puede ir seguido de un color)

-- Desactivar el origen

** Crear una * instancia del objetivo

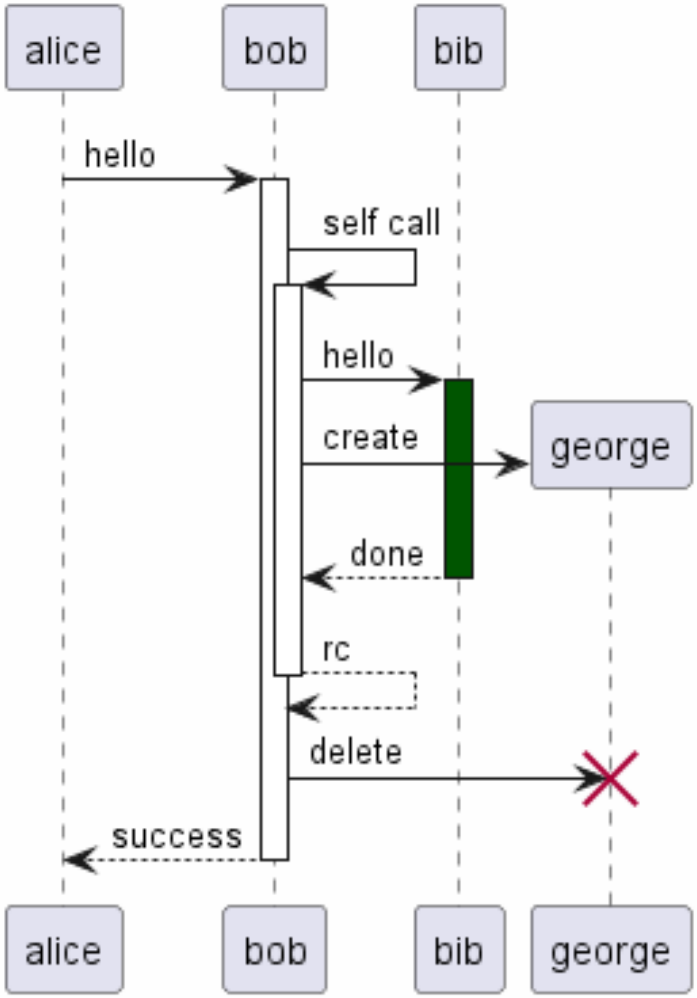
!! Destruir una instancia del objetivo

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Sintaxis abreviada para la activación, desactivación y creación

```
@startuml
alice -> bob ++ : hello
bob -> bob ++ : self call
bob -> bib ++  #005500 : hello
bob -> george ** : create
return done
return rc
bob -> george !! : delete
return success
@enduml
```

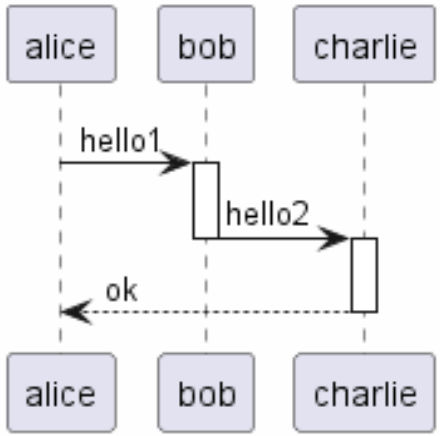


DIAGRAMAS DE COMPORTAMIENTO

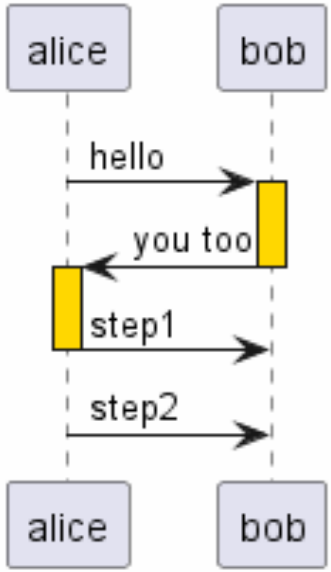
DIAGRAMAS DE SECUENCIA con PlantUML

Luego puedes mezclar activación y desactivación, en la misma línea:

```
@startuml
alice   ->  bob     ++   : hello1
bob     ->  charlie --++ : hello2
charlie --> alice   --   : ok
@enduml
```




```
@startuml
@startuml
alice -> bob   --++ #gold: hello
bob   -> alice --++ #gold: you too
alice -> bob   --: step1
alice -> bob   : step2
@enduml
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

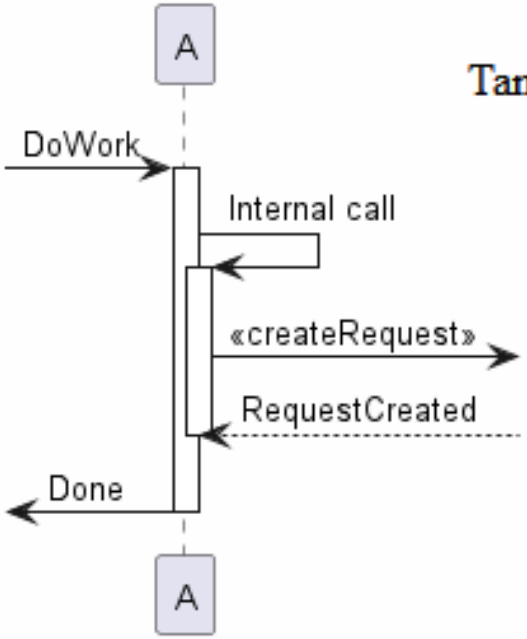
DIAGRAMAS DE SECUENCIA con PlantUML

Mensajes entrantes y salientes

Puedes usar flechas entrantes y salientes si quieres centrarte en una parte del diagrama.

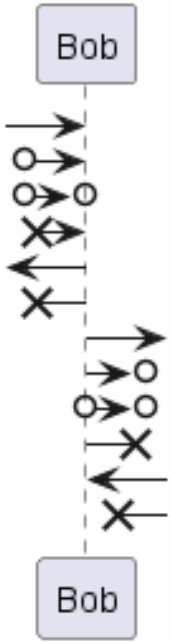
Utilice corchetes para denotar el lado izquierdo " `[` " o el lado derecho " `]` " del diagrama.

```
@startuml
[-> A: DoWork

activate A

A -> A: Internal call
activate A

A ->] : << createRequest >>

A<--] : RequestCreated
deactivate A
[<- A: Done
deactivate A
@enduml
```



También puedes tener la siguiente sintaxis:

```
@startuml
[-> Bob
[o-> Bob
[o->o Bob
[x-> Bob

[<- Bob
[x<- Bob

Bob ->]
Bob ->o]
Bob o->o]
Bob ->x]

Bob <-]
Bob x<-]
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

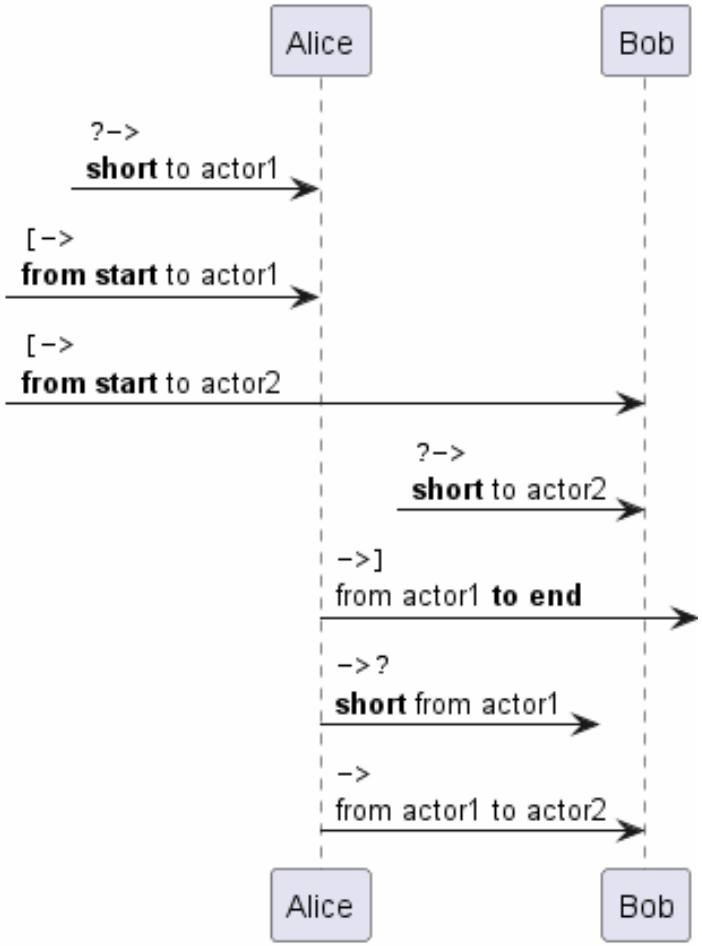
DIAGRAMAS DE SECUENCIA con PlantUML

Flechas cortas para mensajes entrantes y salientes

Flechas cortas ===== para mensajes entrantes y salientes =====

Puedes tener flechas cortas con el uso de ?.

```
@startuml
?-> Alice    : ""?->""\n**short** to actor1
[-> Alice    : ""[->""\n**from start** to actor1
[-> Bob      : ""[->""\n**from start** to actor2
?-> Bob      : ""?->""\n**short** to actor2
Alice ->]    : ""->]""\nfrom actor1 **to end**
Alice ->?    : ""->?""\n**short** from actor1
Alice -> Bob : ""->"" \nfrom actor1 to actor2
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

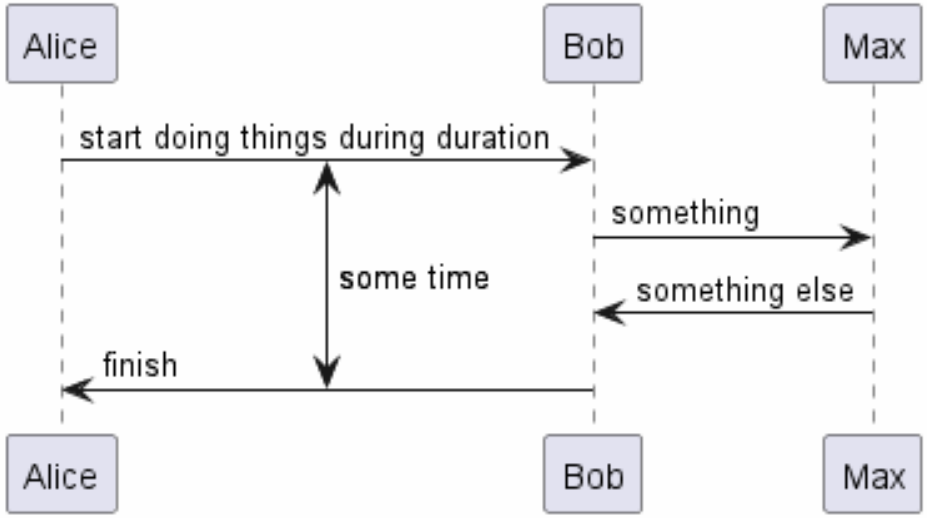
DIAGRAMAS DE SECUENCIA con PlantUML

Anclajes y duración

Anclas ===== y duración =====

Con teoz es posible añadir anclas al diagrama y utilizar las anclas para especificar el tiempo de duración.

```
@startuml
!pragma teoz true

{start} Alice -> Bob : start doing things during duration
Bob -> Max : something
Max -> Bob : something else
{end} Bob -> Alice : finish

{start} <-> {end} : some time

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

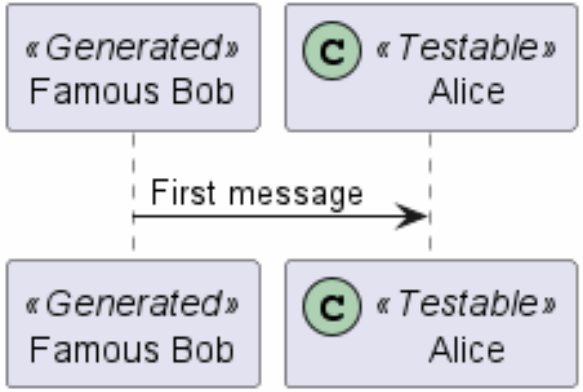
DIAGRAMAS DE SECUENCIA con PlantUML

Estereotipos y marcas

Es posible añadir estereotipos a participantes usando `<<` y `>>`.

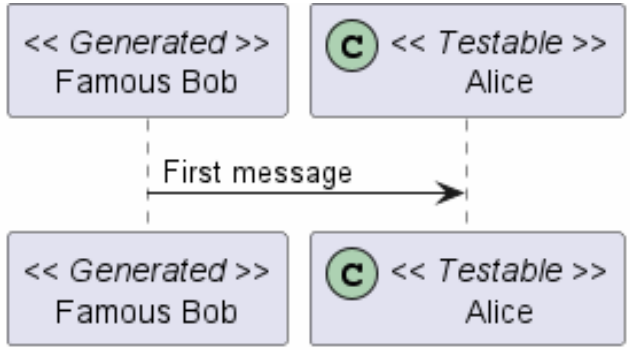
En el estereotipo, puedes añadir un carácter marcado en un círculo coloreado usando la sintaxis `(X,color)`.

```
@startuml

participant "Famous Bob" as Bob << Generated >>
participant Alice << (C,#ADD1B2) Testable >>

Bob->Alice: First message

@enduml
```



Por defecto, guillemet (comillas) son usadas para mostrar el estereotipo. Puedes cambiar este comportamiento usando skinparam guillemet:

```
@startuml

skinparam guillemet false
participant "Famous Bob" as Bob << Generated >>
participant Alice << (C,#ADD1B2) Testable >>

Bob->Alice: First message

@enduml
```

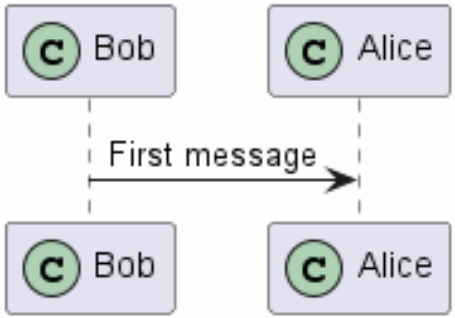


DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

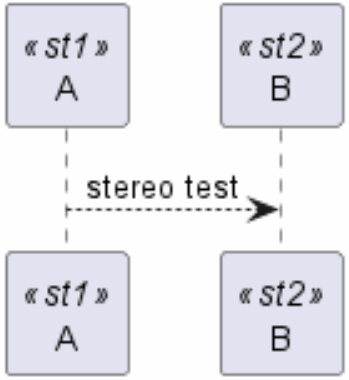
Estereotipos y marcas

```
@startuml

participant Bob << (C,#ADD1B2) >>
participant Alice << (C,#ADD1B2) >>

Bob->Alice: First message

@enduml
```

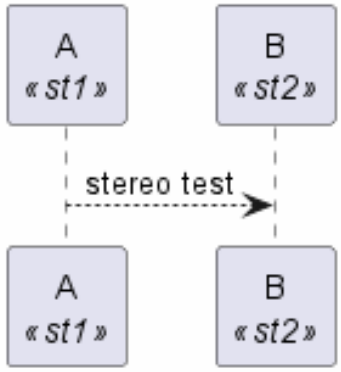


Posición de los estereotipos

```
@startuml
skinparam stereotypePosition top

participant A<<st1>>
participant B<<st2>>
A --> B : stereo test
@enduml
```




```
@startuml
skinparam stereotypePosition bottom

participant A<<st1>>
participant B<<st2>>
A --> B : stereo test
@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

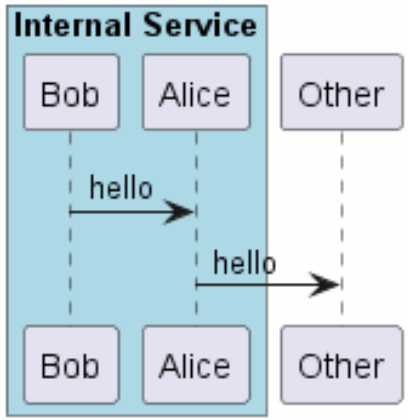
DIAGRAMAS DE SECUENCIA con PlantUML

Entorno de participante

Es posible dibujar una caja alrededor de algunos participantes, usando los comandos `box` y `end box` .

Puedes añadir un título opcional o un color de fondo opcional, después de la palabra reservada `box` .

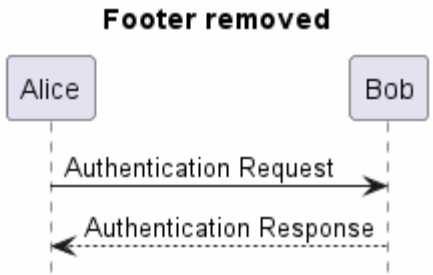
```
@startuml

box "Internal Service" #LightBlue
participant Bob
participant Alice
end box
participant Other

Bob -> Alice : hello
Alice -> Other : hello

@enduml
```



Removiendo pie de página

Puedes usar las palabras reservadas `hide footbox` para remover el pie de página del diagrama.

```
@startuml

hide footbox
title Footer removed

Alice -> Bob: Authentication Request
Bob --> Alice: Authentication Response

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Personalización (Skinparam)

Puedes usar el comando skinparam para cambiar los colores y las fuentes de los dibujos

Puedes usar este comando:

- En la definición del diagrama, como cualquier otro comando,
- En un archivo incluido,
- En un archivo de configuración, proporcionado en la consola de comandos o en el ANT task.

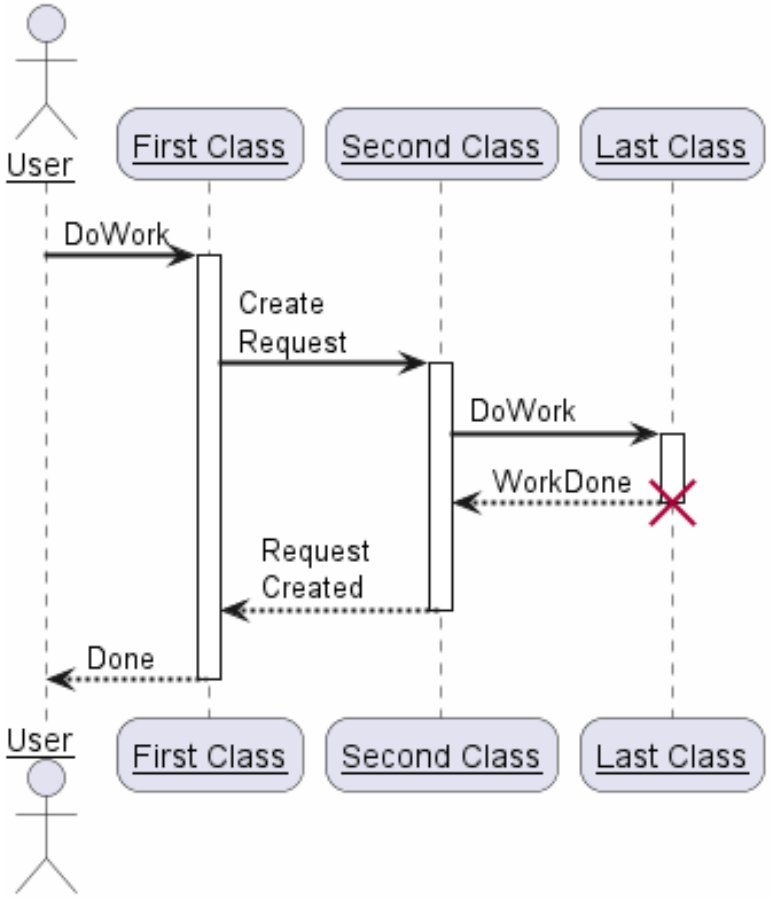
También puedes cambiar otros parámetros de renderización, como se ve en los siguientes ejemplos

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Personalización (Skinparam)

```
@startuml
skinparam sequenceArrowThickness 2
skinparam roundcorner 20
skinparam maxmessagesize 60
skinparam sequenceParticipant underline

actor User
participant "First Class" as A
participant "Second Class" as B
participant "Last Class" as C

User -> A: DoWork
activate A

A -> B: Create Request
activate B

B -> C: DoWork
activate C
C --> B: WorkDone
destroy C

B --> A: Request Created
deactivate B

A --> User: Done
deactivate A

@enduml
```



DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Personalización (Skinparam)

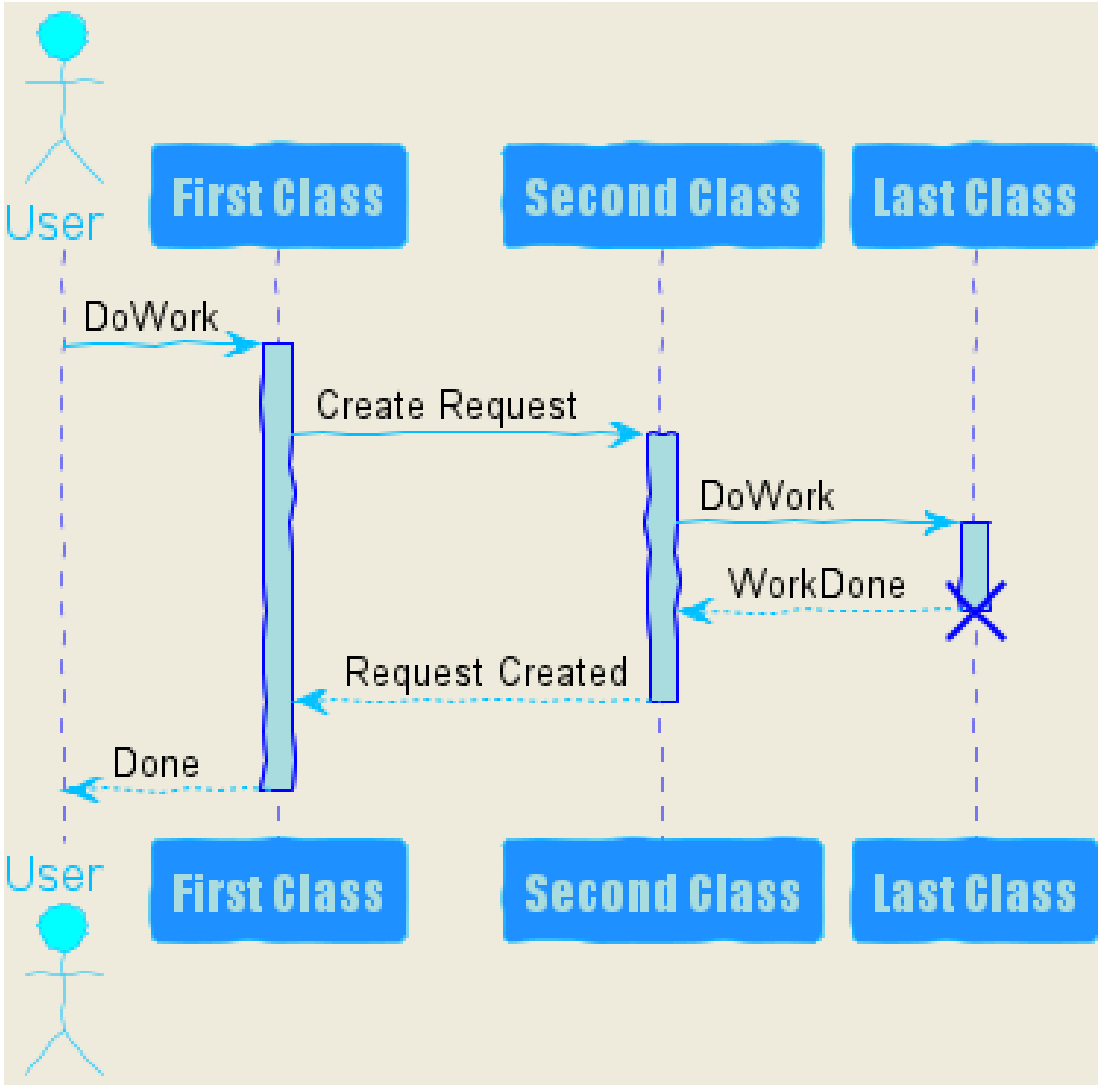
```
@startuml
skinparam backgroundColor #EEEBDC
skinparam handwritten true

skinparam sequence {
ArrowColor DeepSkyBlue
ActorBorderColor DeepSkyBlue
LifeLineBorderColor blue
LifeLineBackgroundColor #A9DCDF

ParticipantBorderColor DeepSkyBlue
ParticipantBackgroundColor DodgerBlue
ParticipantFontName Impact
ParticipantFontSize 17
ParticipantFontColor #A9DCDF

ActorBackgroundColor aqua
ActorFontColor DeepSkyBlue
ActorFontSize 17
ActorFontName Aapex
}
```

```
actor User
participant "First Class" as A
participant "Second Class" as B
participant "Last Class" as C

User -> A: DoWork
activate A

A -> B: Create Request
activate B

B -> C: DoWork
activate C
C --> B: WorkDone
destroy C

B --> A: Request Created
deactivate B

A --> User: Done
deactivate A

@enduml
```

DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Personalización (Skinparam)



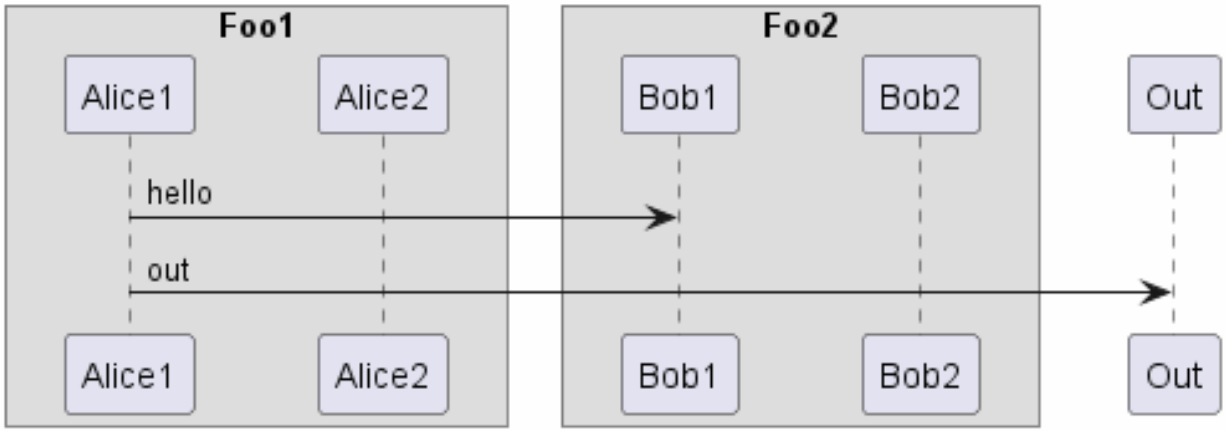
DIAGRAMAS DE COMPORTAMIENTO

DIAGRAMAS DE SECUENCIA con PlantUML

Cambiando el relleno

Es posible ajustar algunos parámetros de relleno

```
@startuml
skinparam ParticipantPadding 20
skinparam BoxPadding 10

box "Foo1"
participant Alice1
participant Alice2
end box
box "Foo2"
participant Bob1
participant Bob2
end box
Alice1 -> Bob1 : hello
Alice1 -> Out : out
@enduml
```



TEMA 5.
DIAGRAMAS DE COMPORTAMIENTO